

计算机基础

Violet

2026-04-14

Chapter 0

目 录

I	版本管理与开发工具
1.	Git 版本控制详解 ····· 6
1.1.	版本控制系统概述 ····· 6
1.2.	Git 核心概念 ····· 7
1.3.	Git 基本操作 ····· 9
1.4.	分支管理 ····· 13
1.5.	远程协作 ····· 16
1.6.	高级操作 ····· 17
1.7.	标签管理 ····· 19
1.8.	Git 工作流 ····· 20
1.9.	.gitignore 配置 ····· 22
1.10.	Git 钩子 (Hooks) ····· 24
1.11.	常见问题与解决方案 ····· 25
1.12.	Git 性能优化 ····· 27
1.13.	Git 最佳实践总结 ····· 28
II	数据库系统
2.	数据库基础理论 ····· 31
2.1.	数据库系统概述与发展历程 ····· 31
2.2.	数据模型 ····· 32
2.3.	数据库体系结构 ····· 35
2.4.	数据独立性 ····· 37
2.5.	数据库管理系统 (DBMS) 的功能与组成 ····· 38
2.6.	事务的基本概念 ····· 40
2.7.	并发控制理论基础 ····· 42
2.8.	数据库安全性与完整性 ····· 45
2.9.	总结 ····· 48
3.	关系型数据库 ····· 50
3.1.	关系模型基础 ····· 50
3.2.	关系代数 ····· 52
3.3.	SQL 语言基础 ····· 56

3.4. 数据查询进阶 60

3.5. 索引技术 65

3.6. 视图与存储过程 70

3.7. 触发器与事件调度 73

3.8. 数据库规范化理论 77

3.9. 反规范化设计与性能优化 80

3.10. 锁机制与并发控制 84

3.11. 事务隔离级别 89

3.12. MVCC 原理 92

3.13. MySQL 架构与存储引擎 97

3.14. PostgreSQL 特性与应用 103

3.15. 关系型数据库性能调优 109

III 并发编程与分布式系统

4. 并发编程基础 117

5. 分布式系统理论 118

5.1. 分布式系统概述与挑战 118

5.2. CAP 定理详解 119

5.3. CAP 权衡与实际应用 122

5.4. BASE 理论 124

5.5. 一致性模型 127

5.6. 分布式事务 129

5.7. 分布式锁实现 134

5.8. Leader 选举算法 138

5.9. 拜占庭将军问题 140

5.10. FL 不可能定理 142

5.11. 总结 144

V 计算机基础综合

6. 正则表达式 147

6.1. 基础概念 147

6.2. 基本语法 147

6.3. 高级特性 150

6.4. 常用正则模式 151

6.5. 各语言中的使用 153

6.6. 性能优化 156

6.7. 调试技巧 157

6.8. 最佳实践总结 158



版本管理与开发工具

Chapter 1

Git 版本控制详解

Git 是目前最流行的分布式版本控制系统，由 Linus Torvalds 于 2005 年创建，用于 Linux 内核开发。它以其速度、数据完整性和对非线性工作流的强大支持而闻名。

NOTE

Git 的核心设计理念是**快照**而非**差异**。每次提交时，Git 都会存储整个项目的快照，如果文件没有变化，则只存储指向之前版本的链接。

1 版本控制系统概述

1.1 集中式 vs 分布式

集中式版本控制 (CVCS)

代表工具：SVN、CVS、Perforce

特点：

- 单一中央服务器存储所有版本历史
- 客户端只获取最新版本
- 必须联网才能进行大部分操作
- 中央服务器是单点故障

优点：

- 权限管理简单
- 学习曲线较低
- 适合小型团队

缺点：

- 网络依赖性强
- 中央服务器故障影响所有人
- 分支和合并操作较慢

分布式版本控制 (DVCS)

代表工具：Git、Mercurial、Bazaar

特点：

- 每个客户端都有完整的仓库副本
- 大多数操作在本地完成
- 支持离线工作
- 无单点故障

优点:

- 速度快（本地操作）
- 支持离线工作
- 分支和合并高效
- 更好的容错性

缺点:

- 学习曲线较陡
- 仓库占用空间较大
- 权限管理复杂

TIP

现代软件开发中，分布式版本控制已成为主流，Git 占据了超过 90% 的市场份额。

1.2 Git 的发展历程

- 2005 年：Linux Torvalds 创建 Git，替代 BitKeeper
- 2008 年：GitHub 上线，推动 Git 普及
- 2010 年后：Git 成为行业标准
- 现在：几乎所有开源项目和大多数企业都使用 Git

2 Git 核心概念

2.1 四个工作区域

Git 有四个主要的工作区域，理解它们是掌握 Git 的关键：

工作区（Working Directory）

- 你正在编辑的文件所在的目录
- 包含项目的当前状态
- 可以随意修改文件

暂存区（Staging Area / Index）

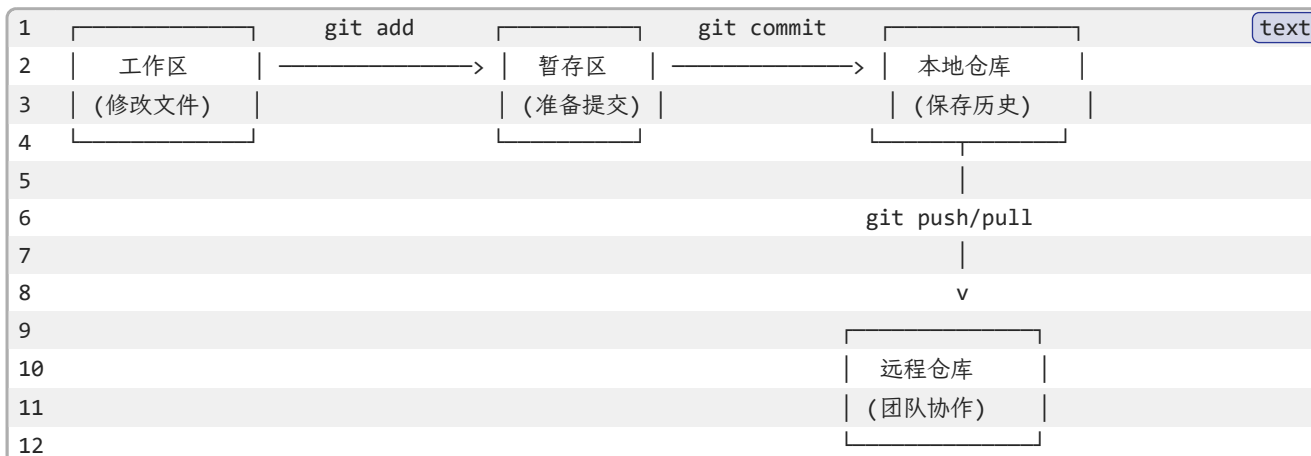
- 也称为“索引”
- 临时存储下次要提交的文件快照
- 通过 `git add` 将文件从工作区添加到暂存区
- 允许你精心准备每次提交的内容

本地仓库（Local Repository）

- 位于 `.git` 目录下
- 存储所有提交历史和元数据
- 是你的完整项目历史
- 通过 `git commit` 将暂存区内容提交到本地仓库

远程仓库 (Remote Repository)

- 存储在远程服务器上的仓库副本
- 如 GitHub、GitLab、Gitee 等平台
- 通过 `git push` 和 `git pull` 同步



CAUTION

新手常犯的错误是直接将工作区的修改 commit，而忽略了 add 步骤。记住：先 add，再 commit！

2.2 文件状态

Git 中的文件有三种状态：

- 已修改 (modified)：文件已被修改但尚未暂存
- 已暂存 (staged)：文件已标记为下次提交的版本
- 已提交 (committed)：文件已安全存储在本地数据库中

2.3 Git 对象类型

Git 底层使用四种对象类型来存储数据：

Blob (二进制大对象)

- 存储文件内容
- 不包含文件名或元数据
- 相同内容的文件只存储一次

Tree (树对象)

- 表示目录结构
- 包含文件名、权限和指向 blob 或其他 tree 的指针
- 类似于文件系统的目录

Commit (提交对象)

- 包含指向 tree 对象的指针
- 包含作者、提交者、时间戳和提交信息
- 包含指向父提交的指针 (形成历史记录)

Tag (标签对象)

- 给特定提交打上永久标签
- 通常用于标记发布版本
- 包含标签名、消息和指向 commit 的指针

TIP

可以使用 `git cat-file -p <hash>` 查看任何 Git 对象的详细内容，深入了解 Git 内部工作原理。

3 Git 基本操作

3.1 初始化仓库

git init

在当前目录创建新的 Git 仓库：

```
1 git init
```

 Bash

会在当前目录创建 `.git` 隐藏目录，包含所有 Git 元数据。

git clone

克隆远程仓库到本地：

```
1 # 克隆仓库
2 git clone https://github.com/user/repo.git
3
4 # 克隆到指定目录
5 git clone https://github.com/user/repo.git my-project
6
7 # 克隆特定分支
8 git clone -b develop https://github.com/user/repo.git
```

 Bash

NOTE

`git clone` 会自动设置 origin 远程仓库，并检出默认分支（通常是 main 或 master）。

3.2 查看状态与历史

git status

显示工作区和暂存区的状态：

```
1 git status
2
3 # 简洁输出
4 git status -s
```

 Bash

输出示例：

```
1 On branch main
```

```
2 Your branch is up to date with 'origin/main'.
3
4 Changes not staged for commit:
5   (use "git add <file>..." to update what will be committed)
6   (use "git restore <file>..." to discard changes in working directory)
7       modified:   README.md
8
9 Untracked files:
10   (use "git add <file>..." to include in what will be committed)
11       new_file.txt
```

git log

查看提交历史：

```
1 # 基本日志
2 git log
3
4 # 一行显示
5 git log --oneline
6
7 # 图形化显示分支
8 git log --graph --oneline --all
9
10 # 显示统计信息
11 git log --stat
12
13 # 显示补丁
14 git log -p
15
16 # 最近n次提交
17 git log -n 5
```

git diff

查看文件差异：

```
1 # 工作区与暂存区的差异
2 git diff
3
4 # 暂存区与最新提交的差异
5 git diff --staged
6
7 # 两个提交之间的差异
8 git diff commit1 commit2
9
10 # 特定文件的差异
11 git diff filename
```

3.3 添加与提交

git add

将文件添加到暂存区：

```
1 # 添加特定文件
2 git add file.txt
3
4 # 添加多个文件
5 git add file1.txt file2.txt
6
7 # 添加所有修改的文件
8 git add .
9
10 # 交互式添加
11 git add -i
12
13 # 添加部分修改 (patch模式)
14 git add -p
```

TIP

使用 `git add -p` 可以逐块审查修改，选择性地暂存部分修改，非常适合精细控制提交内容。

git commit

将暂存区的内容提交到本地仓库：

```
1 # 基本提交
2 git commit -m "feat: add user authentication"
3
4 # 使用编辑器编写详细提交信息
5 git commit
6
7 # 跳过暂存区，直接提交所有修改的文件
8 git commit -a -m "fix: correct typo"
9
10 # 修改最后一次提交
11 git commit --amend
```

提交信息规范 (Conventional Commits)：

```
1 <type>(<scope>): <subject>
2
3 <body>
4
5 <footer>
```

常用 type：

- `feat`：新功能
- `fix`：修复 bug
- `docs`：文档更新
- `style`：代码格式（不影响功能）
- `refactor`：重构

- `test`: 测试相关
- `chore`: 构建过程或辅助工具变动

CAUTION

避免使用模糊的提交信息如“update”、“fix bug”。好的提交信息应该清晰说明做了什么和为什么做。

3.4 撤销操作

git restore

撤销工作区的修改：

```
1 # 撤销单个文件的修改
2 git restore file.txt
3
4 # 撤销所有未暂存的修改
5 git restore .
```

 Bash

git reset

重置暂存区或回退提交：

```
1 # 软重置：保留工作区和暂存区
2 git reset --soft HEAD~1
3
4 # 混合重置（默认）：保留工作区，清空暂存区
5 git reset HEAD~1
6
7 # 硬重置：丢弃工作区和暂存区的修改
8 git reset --hard HEAD~1
9
10 # 重置特定文件
11 git reset HEAD file.txt
```

 Bash**CAUTION**

`git reset --hard` 会永久丢失未提交的修改，使用前务必确认！

git revert

创建新的提交来撤销之前的修改（安全的方式）：

```
1 # 撤销某次提交
2 git revert <commit-hash>
3
4 # 撤销多次提交
5 git revert HEAD~3..HEAD
```

 Bash**TIP**

对于已经 push 到远程的提交，使用 `git revert` 而非 `git reset`，避免破坏共享历史。

4 分支管理

分支是 Git 最强大的特性之一，允许你并行开发多个功能。

4.1 分支基本概念

Git 中的分支本质上是一个指向特定提交的轻量级指针。创建、切换和删除分支都非常快速。

```
1      A---B---C  main
2      \
3      D---E  feature
```

4.2 基本分支操作

查看分支

```
1 # 查看所有本地分支
2 git branch
3
4 # 查看所有分支（包括远程）
5 git branch -a
6
7 # 查看分支及其最后提交
8 git branch -v
9
10 # 查看已合并的分支
11 git branch --merged
12
13 # 查看未合并的分支
14 git branch --no-merged
```

创建分支

```
1 # 创建新分支
2 git branch feature-login
3
4 # 创建并切换到新分支
5 git checkout -b feature-login
6
7 # 基于特定提交创建分支
8 git branch feature-login abc1234
```

切换分支

```
1 # 切换到已有分支
2 git checkout feature-login
3
4 # 或使用新命令
5 git switch feature-login
6
```

```
7 # 切换回上一个分支
8 git checkout -
9 git switch -
```

TIP

Git 2.23+引入了 `git switch` 和 `git restore`，使命令语义更清晰。`git checkout` 仍然可用但功能过于复杂。

删除分支

```
1 # 删除已合并的分支
2 git branch -d feature-login
3
4 # 强制删除未合并的分支
5 git branch -D feature-login
6
7 # 删除远程分支
8 git push origin --delete feature-login
```

 Bash

4.3 合并分支

git merge

将一个分支的更改合并到当前分支：

```
1 # 切换到目标分支
2 git checkout main
3
4 # 合并feature分支
5 git merge feature-login
```

 Bash

合并策略：

- Fast-forward：如果目标分支是源分支的直接祖先，只是移动指针
- Three-way merge：创建一个新的 merge commit
- Squash merge：将所有提交压缩为一个

```
1 # 禁用fast-forward，总是创建merge commit
2 git merge --no-ff feature-login
3
4 # Squash合并
5 git merge --squash feature-login
6 git commit -m "feat: add login feature"
```

 Bash

解决冲突

当两个分支修改了同一文件的同一部分时，会产生冲突：

```
1 # 查看冲突文件
2 git status
3
4 # 手动编辑冲突文件，解决冲突后
5 git add resolved_file.txt
```

 Bash

```
6 git commit
```

冲突标记:

```
1 <<<<<< HEAD
2 你的修改
3 别人的修改
4 >>>>>> feature-branch
```

TIP

使用 `git mergetool` 可以调用可视化的合并工具（如 meld、kdiff3）来解决冲突。

4.4 Rebase（变基）

Rebase 可以将一个分支的提交“重新播放”到另一个分支上，保持线性历史。

```
1 # 将当前分支rebase到main
2 git checkout feature
3 git rebase main
```

Merge vs Rebase:

特性	Merge	Rebase
历史记录	保留完整历史，有 merge commit	线性历史，更清晰
真实性	反映真实的开发过程	重写历史
安全性	安全，不改变历史	可能危险，特别是已 push 的提交
适用场景	公共分支	私有分支

CAUTION

永远不要 rebase 已经 push 到远程的公共分支！这会改写历史，导致其他协作者的仓库出现问题。

交互式 Rebase

可以修改、合并、删除或重新排序提交：

```
1 git rebase -i HEAD~3
```

可用命令：

- `pick`: 保留提交
- `reword`: 保留提交，修改提交信息
- `edit`: 暂停，允许修改提交
- `squash`: 合并到前一个提交
- `fixup`: 类似 squash，但丢弃提交信息
- `drop`: 删除提交

TIP

交互式 rebase 是整理提交历史的强大工具，可以让你的提交历史更加清晰和专业。

5 远程协作

5.1 远程仓库管理

查看远程仓库

```
1 # 查看远程仓库
2 git remote -v
3
4 # 查看远程仓库详细信息
5 git remote show origin
```

 Bash

添加远程仓库

```
1 # 添加远程仓库
2 git remote add origin https://github.com/user/repo.git
3
4 # 添加多个远程仓库
5 git remote add upstream https://github.com/original/repo.git
```

 Bash

重命名和删除远程仓库

```
1 # 重命名
2 git remote rename origin github
3
4 # 删除
5 git remote remove github
```

 Bash

5.2 推送与拉取

git push

将本地提交推送到远程仓库：

```
1 # 推送到默认远程和分支
2 git push
3
4 # 推送到特定远程和分支
5 git push origin main
6
7 # 首次推送，设置上游分支
8 git push -u origin main
9
10 # 强制推送（谨慎使用）
11 git push --force
12
13 # 更安全的强制推送（只覆盖没有其他人推送的部分）
14 git push --force-with-lease
```

 Bash

CAUTION

避免使用 `--force`，优先使用 `--force-with-lease`，它会检查远程分支是否被其他人更新过。

git pull

从远程仓库拉取并合并更改：

```
1 # 拉取并合并
2 git pull
3
4 # 拉取特定分支
5 git pull origin main
6
7 # 拉取但不自动合并
8 git pull --no-commit
9
10 # 使用rebase代替merge
11 git pull --rebase
```

git fetch

仅下载远程更改，不自动合并：

```
1 # 获取所有远程分支的更新
2 git fetch
3
4 # 获取特定远程
5 git fetch origin
6
7 # 获取特定分支
8 git fetch origin main
```

TIP

推荐工作流程：先 `git fetch` 查看远程变化，再决定如何合并，而不是直接使用 `git pull`。

5.3 跟踪分支

```
1 # 设置上游分支
2 git branch --set-upstream-to=origin/main main
3
4 # 简写
5 git branch -u origin/main
6
7 # 查看跟踪关系
8 git branch -vv
```

6 高级操作

6.1 Stash（储藏）

临时保存工作进度，稍后恢复：

```
1 # 储藏当前修改
2 git stash
3
4 # 储藏并包含未跟踪的文件
5 git stash -u
6
7 # 带消息的储藏
8 git stash save "WIP: working on login feature"
9
10 # 查看所有储藏
11 git stash list
12
13 # 应用最近的储藏
14 git stash apply
15
16 # 应用并删除储藏
17 git stash pop
18
19 # 应用特定的储藏
20 git stash apply stash@{2}
21
22 # 删除储藏
23 git stash drop stash@{1}
24
25 # 清空所有储藏
26 git stash clear
```

NOTE

Stash 非常适合需要临时切换分支但又不想提交未完成工作的场景。

6.2 Cherry-pick

选择性地应用某个提交到当前分支：

```
1 # 应用单个提交
2 git cherry-pick abc1234
3
4 # 应用多个提交
5 git cherry-pick abc1234 def5678
6
7 # 应用提交范围
8 git cherry-pick abc1234..def5678
9
10 # 应用但不自动提交
11 git cherry-pick --no-commit abc1234
```

TIP

Cherry-pick 常用于将 bug 修复从开发分支应用到生产分支。

6.3 Reset 深入

```
1 # 软重置：只移动HEAD指针
2 git reset --soft HEAD~1
3
4 # 混合重置：移动HEAD并重置暂存区（默认）
5 git reset HEAD~1
6
7 # 硬重置：移动HEAD、重置暂存区和工作区
8 git reset --hard HEAD~1
9
10 # 重置到特定提交
11 git reset --hard abc1234
12
13 # 重置特定文件
14 git reset HEAD file.txt
```

6.4 Reflog（引用日志）

记录 HEAD 和分支引用的变化历史，是“后悔药”：

```
1 # 查看reflog
2 git reflog
3
4 # 查看特定分支的reflog
5 git reflog show main
6
7 # 恢复到之前的状态
8 git reset --hard HEAD@{3}
9
10 # 恢复丢失的提交
11 git reset --hard abc1234
```

CAUTION

Reflog 只在本地有效，默认保留 90 天。如果执行了 `git gc` 或超过了保留期限，可能无法恢复。

TIP

即使使用了 `git reset --hard` 或删除了分支，只要 reflog 还在，就有可能找回丢失的提交。

7 标签管理

标签用于标记重要的提交点，通常用于发布版本。

7.1 标签类型

轻量标签（Lightweight）

只是一个指向提交的指针：

```
1 git tag v1.0.0
```

 Bash

I 附注标签 (Annotated)

包含完整信息的标签对象：

```
1 git tag -a v1.0.0 -m "Release version 1.0.0"
```

 Bash

TIP

推荐使用附注标签，因为它包含更多信息（打标签的人、日期、消息等）。

7.2 标签操作

```
1 # 查看所有标签
2 git tag
3
4 # 查看匹配模式的标签
5 git tag -l "v1.*"
6
7 # 查看标签详情
8 git show v1.0.0
9
10 # 删除标签
11 git tag -d v1.0.0
12
13 # 推送标签到远程
14 git push origin v1.0.0
15
16 # 推送所有标签
17 git push origin --tags
18
19 # 删除远程标签
20 git push origin --delete v1.0.0
21
22 # 基于特定提交创建标签
23 git tag -a v1.0.0 abc1234 -m "Release version 1.0.0"
```

 Bash

8 Git 工作流

8.1 Git Flow

经典的 Git 分支模型，适合有固定发布周期的项目：

分支类型：

- `main/master`：生产环境代码
- `develop`：开发主分支
- `feature/*`：功能分支，从 develop 分出
- `release/*`：发布分支，从 develop 分出

- `hotfix/*`: 热修复分支, 从 main 分出

工作流程:

1. 从develop创建feature分支
2. 在feature分支上开发
3. 完成后合并回develop
4. 从develop创建release分支进行测试
5. release完成后合并到main和develop
6. 在main上打tag
7. 如有紧急bug, 从main创建hotfix分支
8. hotfix完成后合并到main和develop

NOTE

Git Flow 适合传统软件项目, 但对于持续部署的现代 Web 应用可能过于复杂。

8.2 GitHub Flow

更简单的分支模型, 适合持续部署:

核心原则:

- main 分支始终可部署
- 从 main 创建功能分支
- 频繁推送到远程
- 通过 Pull Request 讨论和审查
- 审查通过后合并到 main
- 立即部署

工作流程:

1. 从main创建分支
2. 提交更改
3. 打开Pull Request
4. 讨论和审查
5. 合并到main
6. 部署

TIP

GitHub Flow 简单灵活, 适合大多数现代软件开发项目, 特别是 SaaS 应用。

8.3 GitLab Flow

结合 Git Flow 和 GitHub Flow 的优点:

特点:

- 以 main 为主开发分支
- 可以有环境分支 (如 production、staging)
- 可以有版本分支 (如 v1.0、v2.0)
- 变更从上游向下游流动

适用场景:

- 需要同时维护多个版本
- 有多个部署环境
- 需要严格的发布管理

8.4 选择建议

工作流	复杂度	适用场景	学习成本
Git Flow	高	传统软件、固定发布周期	高
GitHub Flow	低	持续部署、Web 应用	低
GitLab Flow	中	多版本、多环境	中

TIP

小团队或新项目建议从 GitHub Flow 开始，随着项目复杂度增加再考虑更复杂的工作流。

9 .gitignore 配置

`.gitignore` 文件指定 Git 应该忽略哪些文件和目录。

9.1 基本语法

```
1 # 注释
2
3 # 忽略所有 .log 文件
4 *.log
5
6 # 忽略 node_modules 目录
7 node_modules/
8
9 # 忽略特定文件
10 config/local.json
11
12 # 例外规则（不忽略 important.log）
13 *.log
14 !important.log
15
16 # 忽略所有 txt 文件，除了 readme.txt
17 *.txt
18 !readme.txt
```

9.2 常用忽略规则

```
1 # 依赖目录
2 node_modules/
3 vendor/
4 __pycache__/
5
```

```
6 # 构建输出
7 dist/
8 build/
9 *.o
10 *.pyc
11
12 # 环境变量
13 .env
14 .env.local
15
16 # IDE配置
17 .vscode/
18 .idea/
19 *.swp
20 *.swo
21
22 # 操作系统文件
23 .DS_Store
24 Thumbs.db
25
26 # 日志文件
27 *.log
28 logs/
29
30 # 临时文件
31 tmp/
32 temp/
```

9.3 全局.gitignore

创建全局忽略文件，对所有仓库生效：

```
1 # 创建全局gitignore
2 git config --global core.excludesfile ~/.gitignore_global
3
4 # 编辑文件
5 vim ~/.gitignore_global
```

 Bash

TIP

将 IDE 配置、操作系统文件等个人偏好设置放在全局.gitignore 中，避免污染项目级别的配置。

9.4 .gitignore 最佳实践

- 在项目根目录放置 `.gitignore`
- 使用在线生成器（如 gitignore.io）生成模板
- 定期审查和更新
- 不要忽略必要的配置文件
- 团队成员应保持一致的忽略规则

10 Git 钩子 (Hooks)

Git 钩子是脚本，在特定事件发生时自动执行。

10.1 钩子类型

客户端钩子

- `pre-commit`: 提交前执行
- `prepare-commit-msg`: 准备提交消息时
- `commit-msg`: 验证提交消息
- `post-commit`: 提交后执行
- `pre-push`: 推送前执行

服务端钩子

- `pre-receive`: 接收推送前
- `update`: 更新引用前
- `post-receive`: 接收推送后

10.2 使用示例

pre-commit 钩子

在 `.git/hooks/pre-commit` 中:

```
1 #!/bin/bash
2
3 # 检查是否有调试代码
4 if grep -r "console.log" --include="*.js" .; then
5     echo "Error: Found console.log statements"
6     exit 1
7 fi
8
9 # 运行代码格式化
10 npm run lint
11
12 # 运行测试
13 npm test
```

```
1 # 使钩子可执行
2 chmod +x .git/hooks/pre-commit
```

NOTE

Git 钩子不会被 clone，需要在每个仓库中单独设置。可以使用工具如 Husky (Node.js) 来自动化这个过程。

commit-msg 钩子

验证提交消息格式:

```
1  #!/bin/bash
2
3  commit_msg=$(cat $1)
4
5  # 检查是否符合Conventional Commits规范
6  if ! echo "$commit_msg" | grep -qE "^(feat|fix|docs|style|refactor|test|chore)(\(.+\))?: .+";
   then
7      echo "Error: Commit message does not follow Conventional Commits format"
8      echo "Format: <type>(<scope>): <subject>"
9      exit 1
10 fi
```

10.3 Husky (推荐)

对于 Node.js 项目，使用 Husky 管理 Git 钩子：

```
1  # 安装Husky
2  npm install husky --save-dev
3  npx husky install
4
5  # 添加pre-commit钩子
6  npx husky add .husky/pre-commit "npm test"
7
8  # 添加commit-msg钩子
9  npx husky add .husky/commit-msg 'npx --no-install commitlint --edit "$1"'
```

TIP

使用 Husky + lint-staged 可以在提交前自动运行 linter 和测试，确保代码质量。

11 常见问题与解决方案

11.1 合并冲突解决

预防冲突

- 频繁 pull 远程更改
- 小而频繁的提交
- 清晰的沟通（谁在修改什么）
- 使用功能分支

解决冲突的步骤

```
1  # 1. 查看冲突文件
2  git status
3
4  # 2. 打开冲突文件，手动解决冲突
5  # 查找 <<<<<<<, =====, >>>>>>> 标记
6
7  # 3. 标记冲突已解决
```

```
8  git add resolved_file.txt
9
10 # 4. 完成合并
11 git commit
```

使用工具

```
1 # 使用内置合并工具
2 git mergetool
3
4 # 配置喜欢的合并工具
5 git config --global merge.tool meld
```

 Bash

11.2 修改提交历史

修改最后一次提交

```
1 # 修改提交消息
2 git commit --amend -m "New commit message"
3
4 # 添加遗漏的文件
5 git add forgotten_file.txt
6 git commit --amend --no-edit
```

 Bash

CAUTION

只修改还未 push 的提交！修改已 push 的提交会影响其他协作者。

修改多次提交

使用交互式 rebase：

```
1 git rebase -i HEAD~3
```

 Bash

然后根据需要选择 `reword`、`edit`、`squash` 等操作。

11.3 恢复丢失的提交

```
1 # 1. 查看reflog
2 git reflog
3
4 # 2. 找到丢失的提交的hash
5 git show abc1234
6
7 # 3. 恢复提交
8 git cherry-pick abc1234
9 # 或
10 git reset --hard abc1234
```

 Bash

11.4 清理仓库

```
1 # 删除未跟踪的文件
2 git clean -n # 预览
3 git clean -f # 执行
4
5 # 删除未跟踪的目录
6 git clean -fd
7
8 # 优化仓库
9 git gc
10
11 # 清理远程追踪分支
12 git remote prune origin
```

12 Git 性能优化

12.1 大仓库优化

浅克隆

```
1 # 只克隆最近的历史
2 git clone --depth 1 https://github.com/user/repo.git
3
4 # 克隆特定深度的历史
5 git clone --depth 50 https://github.com/user/repo.git
```

稀疏检出

```
1 # 只检出特定目录
2 git clone --sparse https://github.com/user/repo.git
3 cd repo
4 git sparse-checkout set src/docs
```

12.2 加速 Git 操作

启用 FSMonitor

```
1 git config core.fsmonitor true
```

使用 SSD

Git 操作大量涉及小文件读写，SSD 能显著提升性能。

增加内存缓存

```
1 git config --global core.preloadIndex true
2 git config --global core.unpackLimit 10000
```

12.3 减小仓库大小

```
1 # 清理不必要的对象
2 git gc --prune=now --aggressive
3
4 # 移除大文件历史（使用git-filter-repo）
5 pip install git-filter-repo
6 git filter-repo --path large_file.bin --invert-paths
```

CAUTION

修改历史后需要强制推送，并确保团队成员知道如何处理。

12.4 分包（Submodules）

将大型依赖作为子模块：

```
1 # 添加子模块
2 git submodule add https://github.com/user/lib.git libs/mylib
3
4 # 初始化子模块
5 git submodule init
6 git submodule update
7
8 # 克隆包含子模块的仓库
9 git clone --recursive https://github.com/user/repo.git
```

TIP

对于现代项目，考虑使用包管理器（npm、pip、maven 等）代替 submodules，更简单易用。

13 Git 最佳实践总结

13.1 提交规范

- 小而频繁的提交
- 清晰的提交信息
- 一个提交只做一件事
- 遵循 Conventional Commits 规范

13.2 分支策略

- 使用有意义的分支名
- 及时删除已合并的分支
- 定期 rebase 以保持历史清晰
- 不要 rebase 公共分支

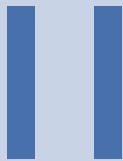
13.3 协作礼仪

- 经常 push 和 pull
- 代码审查后再合并
- 尊重他人的工作
- 清晰的 PR 描述

13.4 安全措施

- 使用 SSH 密钥而非 HTTPS 密码
- 启用双因素认证
- 定期备份重要仓库
- 谨慎使用 force push

本章完



数据库系统

2.	数据库基础理论 ·····	31
3.	关系型数据库 ·····	50

数据库基础理论

NOTE

1 数据库系统概述与发展历程

I 文件系统阶段（1950s-1960s）

特点：

- 数据可以长期保存
- 文件系统管理数据
- 数据共享性差
- 数据独立性差

问题：数据冗余、不一致、难以维护

I 数据库系统阶段（1960s 至今）

特点：

- 数据结构化
- 数据共享性高
- 数据独立性高
- 统一的数据控制

优势：减少冗余、保持一致、易于维护

TIP

数据库系统的出现是数据管理技术的革命，解决了文件系统的诸多问题。

1.3 数据库系统的特点

1. 数据结构化：整体结构化，不仅内部结构化，整体也结构化
2. 数据共享性高：多个用户、多个应用可以同时使用
3. 数据独立性高：物理独立性和逻辑独立性
4. 统一的数据控制：安全性、完整性、并发控制、恢复

2 数据模型

数据模型是对现实世界数据特征的抽象，是数据库系统的核心和基础。

2.1 数据模型的要素

1. 数据结构：描述系统的静态特性
2. 数据操作：描述系统的动态特性
3. 数据约束：完整性规则集合

2.2 层次模型

用树形结构表示实体及其联系。

特点：

- 有且只有一个根节点
- 其他节点有且只有一个父节点
- 适合表示一对多关系

1	学校
2	/ \

text

3	学院	学院
4	/ \	
5	专业	专业 专业

优点:

- 结构简单
- 查询效率高

缺点:

- 只能表示一对多关系
- 插入删除操作复杂
- 查询子节点必须经过父节点

NOTE

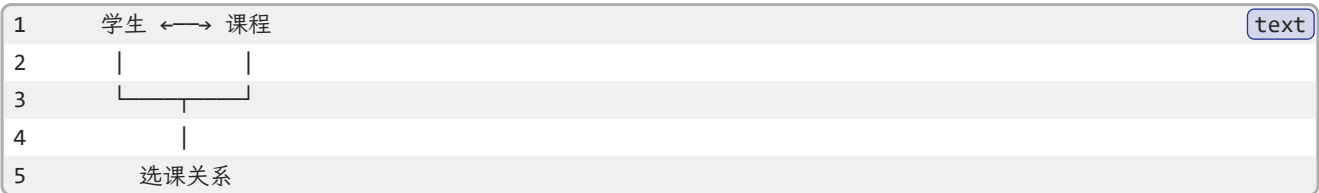
层次模型的代表系统是 IBM 的 IMS (Information Management System)。

2.3 网状模型

用图结构表示实体及其联系。

特点:

- 允许一个以上节点无双亲
- 一个节点可以有多于一个双亲
- 可以表示多对多关系



优点:

- 能直接描述现实世界
- 存取效率高

缺点:

- 结构复杂
- DDL 和 DML 语言复杂
- 用户不易掌握

2.4 关系模型（最重要）

用二维表结构表示实体及其联系。

基本概念:

- 关系 (Relation) : 一张二维表
- 元组 (Tuple) : 表中的一行
- 属性 (Attribute) : 表中的一列
- 域 (Domain) : 属性的取值范围
- 码 (Key) : 唯一标识元组的属性或属性组

- 关系模式：关系的结构描述

示例：

1	学生表：				text
2					
3	学号	姓名	年龄	系别	
4					
5	001	张三	20	计算机	
6	002	李四	21	数学	
7	003	王五	19	物理	
8					

优点：

- 结构简单清晰
- 理论基础坚实（关系代数、关系演算）
- 数据独立性高
- 使用方便

缺点：

- 查询效率可能较低
- 复杂查询需要连接操作

TIP

关系模型是目前最主流的数据库模型，MySQL、PostgreSQL、Oracle 等都是关系型数据库。

2.5 面向对象模型

将面向对象的概念引入数据库。

特点：

- 封装性
- 继承性
- 多态性
- 支持复杂数据类型

适用场景：

- CAD/CAM 系统
- 多媒体数据库
- 地理信息系统

2.6 ER 模型（实体-联系模型）

用于概念设计的工具，独立于具体的 DBMS。

基本要素：

- 实体（Entity）：客观存在并可相互区别的事物
- 属性（Attribute）：实体所具有的某一特性
- 联系（Relationship）：实体之间的关联

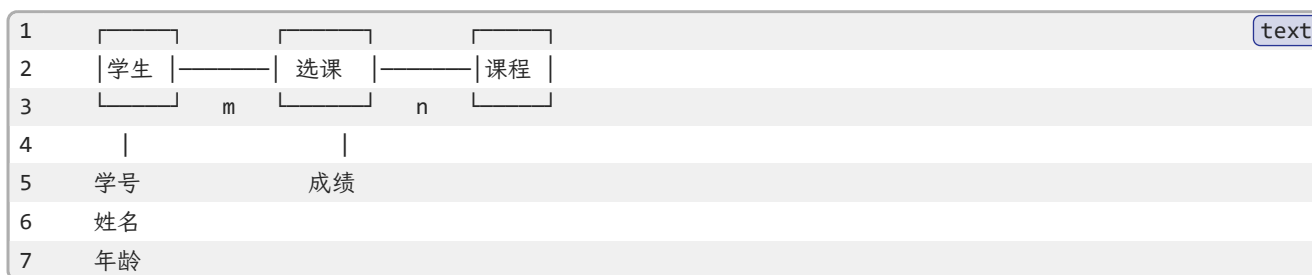
联系的类型：

- 一对一 (1:1) : 一个实体最多与另一个实体的一个实例相关联
- 一对多 (1:n) : 一个实体可以与多个另一类实体相关联
- 多对多 (m:n) : 两个实体集之间是多对多关系

ER 图符号:

- 1 矩形: 实体
- 2 椭圆: 属性
- 3 菱形: 联系
- 4 直线: 连接

示例：



NOTE

ER 模型是数据库设计的重要工具，帮助我们从现实世界抽象出数据模型。

3 数据库体系结构

3.1 三级模式结构

数据库系统的三级模式结构提供了数据抽象的三个级别。

外模式 (External Schema)

定义：用户能看到和使用的局部数据的逻辑结构和特征的描述。

特点:

- 也称子模式或用户模式
- 一个数据库可以有多个外模式
- 是用户的数据视图
- 保证数据安全

模式 (Schema)

定义：数据库中全体数据的逻辑结构和特征的描述。

特点:

- 也称逻辑模式
- 一个数据库只有一个模式
- 是所有用户的公共数据视图
- 不涉及物理存储细节

内模式 (Internal Schema)

定义：数据在数据库系统内部的表示方式。

特点：

- 也称存储模式
- 一个数据库只有一个内模式
- 涉及物理存储结构
- 包括索引、压缩、加密等

1			text
2	外模式 1	← 用户A视图	
3			
4	外模式 2	← 用户B视图	
5			
6	外模式 n	← 用户N视图	
7			
8	模式	← 全局逻辑视图	
9			
10	内模式	← 物理存储视图	
11			

3.2 两级映像

外模式/模式映像

定义：定义外模式与模式之间的对应关系。

作用：

- 当模式改变时，修改映像
- 外模式可以保持不变
- 保证数据的逻辑独立性

模式/内模式映像

定义：定义模式与内模式之间的对应关系。

作用：

- 当内模式改变时，修改映像
- 模式可以保持不变
- 保证数据的物理独立性

1	用户请求	text
2	↓	
3	外模式	
4	↓	
5	外模式/模式映像 ← 逻辑独立性	
6	↓	
7	模式	
8	↓	
9	模式/内模式映像 ← 物理独立性	
10	↓	

11	内模式
12	↓
13	物理存储

TIP

两级映像是数据库系统实现数据独立性的关键机制。

4 数据独立性

数据独立性是数据库系统的重要目标，包括物理独立性和逻辑独立性。

4.1 物理独立性

定义：用户的应用程序与数据库中数据的物理存储是相互独立的。

含义：

- 当数据的物理存储改变了（如更换存储设备、调整索引策略）
- 应用程序不需要改变

实现：通过模式/内模式映像

示例：

1	原存储：数据按姓名排序
2	新存储：数据按学号排序 + B+树索引
3	
4	应用程序：SELECT * FROM students WHERE id = '001'
5	结果：无需修改

4.2 逻辑独立性

定义：用户的应用程序与数据库的逻辑结构是相互独立的。

含义：

- 当数据的逻辑结构改变了（如增加字段、拆分表）
- 应用程序不需要改变

实现：通过外模式/模式映像

示例：

1	原模式：students(id, name, age, dept)
2	新模式：students(id, name, age), departments(dept_id, dept_name)
3	
4	外模式：保持students(id, name, age, dept)不变
5	应用程序：无需修改

CAUTION

逻辑独立性比物理独立性更难实现，因为应用程序通常依赖于特定的数据结构。

4.3 数据独立性的意义

- 1. 降低维护成本：数据结构变化不影响应用
- 2. 提高开发效率：开发者关注业务逻辑
- 3. 增强系统灵活性：易于优化和调整
- 4. 保护投资：延长应用程序寿命

独立性类型	变化内容	影响范围	实现机制
物理独立性	存储结构、索引	最小	模式/内模式映像
逻辑独立性	逻辑结构、表结构	较小	外模式/模式映像

5 数据库管理系统（DBMS）的功能与组成

5.1 DBMS 的主要功能

数据定义功能（DDL）

提供数据定义语言，定义数据库结构：

```
1  -- 创建数据库
2  CREATE DATABASE university;
3
4  -- 创建表
5  CREATE TABLE students (
6      id VARCHAR(10) PRIMARY KEY,
7      name VARCHAR(50) NOT NULL,
8      age INT,
9      dept VARCHAR(50)
10 );
11
12 -- 修改表结构
13 ALTER TABLE students ADD COLUMN email VARCHAR(100);
14
15 -- 删除表
16 DROP TABLE students;
```

数据操纵功能（DML）

提供数据操纵语言，实现对数据的操作：

```
1  -- 插入数据
2  INSERT INTO students VALUES ('001', '张三', 20, '计算机');
3
4  -- 查询数据
5  SELECT * FROM students WHERE dept = '计算机';
6
7  -- 更新数据
8  UPDATE students SET age = 21 WHERE id = '001';
9
10 -- 删除数据
```

```
11 DELETE FROM students WHERE id = '001';
```

数据库运行管理功能

- 并发控制：确保多用户同时访问时的数据一致性
- 事务管理：保证事务的 ACID 特性
- 安全性检查：用户身份验证和权限控制
- 完整性检查：确保数据符合约束条件

数据库的建立和维护功能

- 初始数据装入：批量导入数据
- 数据库转储：定期备份
- 数据库恢复：故障后恢复数据
- 数据库重组：优化存储结构
- 性能监控：分析系统性能

数据组织、存储和管理

- 索引管理：B+树、哈希索引等
- 数据压缩：节省存储空间
- 数据加密：保护敏感数据
- 缓存管理：提高访问速度

5.2 DBMS 的组成



26	
27	
28	磁盘存储
29	

NOTE

不同 DBMS 的具体实现可能有所不同，但基本功能模块相似。

6 事务的基本概念

事务是数据库操作的基本单位，是用户定义的一个操作序列。

6.1 事务的定义

事务 (Transaction)：作为单个逻辑工作单元执行的一系列操作，要么完全执行，要么完全不执行。

示例：银行转账

```
1 BEGIN TRANSACTION;
2
3 -- 从账户A扣款
4 UPDATE accounts SET balance = balance - 1000 WHERE account_id = 'A';
5
6 -- 向账户B存款
7 UPDATE accounts SET balance = balance + 1000 WHERE account_id = 'B';
8
9 COMMIT; -- 提交事务
```

如果任何一步失败：

```
1 ROLLBACK; -- 回滚事务
```

6.2 ACID 特性

事务必须满足 ACID 四个特性：

原子性 (Atomicity)

定义：事务是一个不可分割的工作单位，要么全部执行，要么全部不执行。

实现：通过日志和回滚机制

```
1 事务执行过程:
2 开始 → 操作1 → 操作2 → ... → 操作n → 提交
3      ↓
4      如果失败
5      ↓
6      回滚到开始状态
```

一致性 (Consistency)

定义：事务执行前后，数据库必须处于一致状态。

含义：

- 事务不能破坏数据库的完整性约束
- 从一个一致状态变换到另一个一致状态

示例：

- 1 转账前：A账户1000元，B账户2000元，总计3000元
- 2 转账后：A账户0元，B账户3000元，总计3000元
- 3
- 4 总额保持不变，满足一致性

隔离性 (Isolation)

定义：多个事务并发执行时，一个事务的执行不应影响其他事务。

隔离级别：

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED	可能	可能	可能
READ COMMITTED	不可能	可能	可能
REPEATABLE READ	不可能	不可能	可能
SERIALIZABLE	不可能	不可能	不可能

说明：

- 脏读：读取了未提交的数据
- 不可重复读：同一事务中多次读取结果不同
- 幻读：同一查询返回不同行数

TIP

隔离级别越高，并发性越差。需要根据业务需求选择合适的隔离级别。

持久性 (Durability)

定义：一旦事务提交，其对数据库的改变就是永久的。

实现：

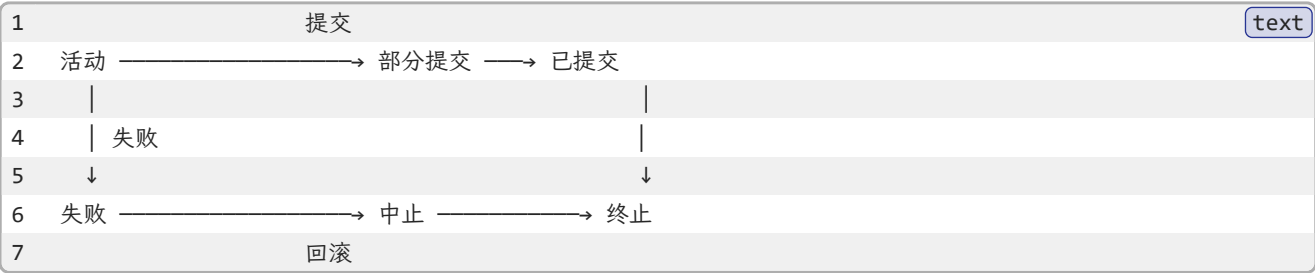
- 写-ahead 日志 (WAL)
- 定期 checkpoint
- 数据冗余备份

- 1 事务提交流程：
- 2 1. 写入日志
- 3 2. 刷新日志到磁盘
- 4 3. 提交事务
- 5 4. 异步写入数据文件
- 6
- 7 即使系统在步骤4之前崩溃，也可以通过日志恢复

CAUTION

ACID 特性是传统关系型数据库的核心保证。NoSQL 数据库通常会放宽某些特性以换取性能和可扩展性。

6.3 事务的状态



状态说明：

- 活动（Active）：事务正在执行
- 部分提交（Partially Committed）：最后一条语句执行完毕
- 失败（Failed）：正常执行无法继续
- 中止（Aborted）：事务回滚，数据库恢复到事务开始前
- 已提交（Committed）：事务成功完成
- 终止（Terminated）：事务结束

7 并发控制理论基础

当多个事务同时执行时，需要并发控制来保证数据一致性。

7.1 并发带来的问题

丢失修改（Lost Update）

1 时间	事务T1	事务T2
2		
3 t1	读取A=100	
4 t2		读取A=100
5 t3	$A = A - 20 = 80$	
6 t4		$A = A - 30 = 70$
7 t5	写入A=80	
8 t6		写入A=70 ← T1的修改丢失

脏读（Dirty Read）

1 时间	事务T1	事务T2
2		
3 t1	读取A=100	
4 t2	$A = A - 20 = 80$	
5 t3	写入A=80	
6 t4		读取A=80 ← 读到未提交数据
7 t5	ROLLBACK	← T1回滚
8 t6		A恢复为100，但T2已基于80计算

不可重复读（Non-repeatable Read）

1 时间	事务T1	事务T2
------	------	------

2		
3	t1	读取A=100
4	t2	$A = A + 50 = 150$
5	t3	写入A=150
6	t4	COMMIT
7	t5	读取A=150 ← 两次读取结果不同

I 幻读 (Phantom Read)

1	时间	事务T1	事务T2	text
2				
3	t1	SELECT * FROM users		
4		WHERE age > 20		
5		返回2条记录		
6	t2		INSERT INTO users	
7			VALUES ('新用户', 25)	
8	t3		COMMIT	
9	t4	SELECT * FROM users		
10		WHERE age > 20		
11		返回3条记录		← 出现"幻影"行

7.2 封锁协议

锁是实现并发控制的主要技术。

I 锁的类型

排他锁 (X 锁, 写锁) :

- 事务 T 对数据 A 加 X 锁后, 其他事务不能再对 A 加任何锁
- 直到 T 释放 A 上的锁

共享锁 (S 锁, 读锁) :

- 事务 T 对数据 A 加 S 锁后, 其他事务只能再对 A 加 S 锁
- 直到 T 释放 A 上的 S 锁

当前锁	请求 S 锁	请求 X 锁
无锁	成功	成功
S 锁	成功	等待
X 锁	等待	等待

I 三级封锁协议

一级封锁协议:

- 事务 T 修改数据 R 之前必须先对其加 X 锁
- 直到事务结束才释放

解决的问题: 丢失修改

二级封锁协议:

- 在一级基础上

- 读取数据 R 之前必须先对其加 S 锁
- 读完后即可释放 S 锁

解决的问题：丢失修改、脏读

三级封锁协议：

- 在一级基础上
- 读取数据 R 之前必须先对其加 S 锁
- 直到事务结束才释放

解决的问题：丢失修改、脏读、不可重复读

NOTE

封锁级别越高，并发性越差。需要根据实际需求选择合适的封锁协议。

两段锁协议（2PL）

定义：所有事务必须分两个阶段对数据项加锁和解锁。

扩展阶段（Growing Phase）：

- 可以获得锁
- 不能释放锁

收缩阶段（Shrinking Phase）：

- 可以释放锁
- 不能获得新锁

1 事务T:	text
2 LOCK-S(A)	← 扩展阶段
3 LOCK-X(B)	
4 LOCK-S(C)	
5 ——— 锁点 (Lock Point) ———	
6 UNLOCK(A)	← 收缩阶段
7 UNLOCK(B)	
8 UNLOCK(C)	

定理：如果所有事务都遵循 2PL 协议，则它们的任何调度都是可串行化的。

CAUTION

2PL 保证了可串行化，但可能导致死锁。

7.3 死锁处理

死锁预防

一次封锁法：

- 事务必须一次性申请所有需要的锁
- 缺点：资源利用率低

顺序封锁法：

- 预先规定锁的顺序
- 事务必须按顺序申请锁

死锁检测与解除

超时法：

- 设置超时时间
- 超过时间则认为死锁

等待图法：

- 构建等待图
- 检测图中是否有环

解除方法：

- 选择一个或多个事务回滚
- 通常选择代价最小的事务

TIP

现代 DBMS 通常采用死锁检测和自动回滚机制。

8 数据库安全性与完整性

8.1 数据库安全性

保护数据库以防止非法使用所造成的数据泄露、更改或破坏。

安全威胁

- 非授权用户对数据库的恶意存取和破坏
- 敏感数据被泄露
- 完整性遭到破坏
- 可用性遭到破坏

安全性控制措施

用户标识和鉴定

```
1 -- 创建用户
2 CREATE USER alice IDENTIFIED BY 'password123';
3
4 -- 修改密码
5 ALTER USER alice IDENTIFIED BY 'new_password';
6
7 -- 删除用户
8 DROP USER alice;
```

存取控制

自主存取控制（DAC）：

```
1 -- 授予权限
2 GRANT SELECT, INSERT ON students TO alice;
```

```
3 GRANT ALL PRIVILEGES ON database.* TO admin;
4
5 -- 收回权限
6 REVOKE INSERT ON students FROM alice;
7
8 -- 查看权限
9 SHOW GRANTS FOR alice;
```

强制存取控制（MAC）：

- 每个数据对象被标以一定的密级
- 每个用户也被授予某一个级别的许可证
- 只有当用户的许可证级别大于等于数据对象的密级时，才能访问

视图机制

```
1 -- 创建视图，隐藏敏感信息
2 CREATE VIEW student_public AS
3 SELECT id, name, dept
4 FROM students;
5 -- 不包含age等敏感字段
6
7 -- 用户只能访问视图
8 GRANT SELECT ON student_public TO public_user;
```

审计

```
1 -- 开启审计
2 AUDIT SELECT, INSERT, UPDATE, DELETE ON students;
3
4 -- 查看审计日志
5 SELECT * FROM audit_log;
```

审计内容：

- 谁（哪个用户）
- 什么时候
- 做了什么操作
- 操作是否成功

数据加密

存储加密：

- 透明数据加密（TDE）
- 列级加密

传输加密：

- SSL/TLS

```
1 -- MySQL启用SSL
2 GRANT ALL PRIVILEGES ON *.* TO 'user'@'%' REQUIRE SSL;
```

NOTE

安全性是多层次的，需要从网络、系统、数据库、应用等多个层面综合考虑。

8.2 数据库完整性

防止数据库中存在不符合语义的数据，防止错误信息的输入和输出。

完整性约束类型

实体完整性

定义：主键不能为空且必须唯一。

```
1 CREATE TABLE students (  
2     id VARCHAR(10) PRIMARY KEY, -- 主键约束  
3     name VARCHAR(50) NOT NULL,  
4     age INT  
5 );
```

参照完整性

定义：外键必须是另一个表的主键的有效值，或者是 NULL。

```
1 CREATE TABLE enrollments (  
2     student_id VARCHAR(10),  
3     course_id VARCHAR(10),  
4     grade DECIMAL(3, 1),  
5     PRIMARY KEY (student_id, course_id),  
6     FOREIGN KEY (student_id) REFERENCES students(id)  
7         ON DELETE CASCADE -- 级联删除  
8         ON UPDATE CASCADE, -- 级联更新  
9     FOREIGN KEY (course_id) REFERENCES courses(id)  
10 );
```

参照动作：

- **CASCADE**：级联删除/更新
- **SET NULL**：设为 NULL
- **RESTRICT**：拒绝操作
- **NO ACTION**：不采取行动

用户定义的完整性

域约束：

```
1 CREATE TABLE employees (  
2     id INT PRIMARY KEY,  
3     name VARCHAR(50),  
4     age INT CHECK (age >= 18 AND age <= 65),  
5     gender CHAR(1) CHECK (gender IN ('M', 'F')),  
6     salary DECIMAL(10, 2) CHECK (salary > 0)  
7 );
```

断言 (Assertion) :

```
1  -- 并非所有DBMS都支持
2  CREATE ASSERTION salary_check
3  CHECK (
4      NOT EXISTS (
5          SELECT * FROM employees
6          WHERE salary > (
7              SELECT AVG(salary) * 5 FROM employees
8          )
9      )
10 );
```

触发器 (Trigger) :

```
1  CREATE TRIGGER check_salary
2  BEFORE INSERT OR UPDATE ON employees
3  FOR EACH ROW
4  BEGIN
5      IF NEW.salary < 0 THEN
6          SIGNAL SQLSTATE '45000'
7          SET MESSAGE_TEXT = 'Salary cannot be negative';
8      END IF;
9  END;
```

TIP

完整性约束应该在数据库层面定义，而不是仅在应用层检查，以保证数据的一致性。

完整性检查时机

- 立即检查：每条语句执行后立即检查
- 延迟检查：事务提交时检查

```
1  -- 设置延迟检查
2  SET CONSTRAINTS ALL DEFERRED;
3
4  -- 事务中的操作
5  BEGIN;
6  INSERT INTO orders VALUES (...);
7  UPDATE inventory SET stock = stock - 1 WHERE ...;
8  COMMIT; -- 此时检查完整性
```

9 总结

数据库基础理论涵盖了以下核心内容：

- 数据模型：层次、网状、关系、面向对象模型，其中关系模型最为重要
- 体系结构：三级模式（外模式、模式、内模式）和两级映像
- 数据独立性：物理独立性和逻辑独立性
- DBMS 功能：数据定义、操纵、运行管理、建立维护
- 事务管理：ACID 特性是数据库可靠性的保证
- 并发控制：封锁协议、两段锁、死锁处理

- 安全性与完整性：多层次的安全控制和完整性约束



下一章将深入探讨关系数据库标准语言 SQL 的使用。

Chapter 3

关系型数据库

关系型数据库是目前应用最广泛的数据库类型，基于关系模型理论，使用 SQL 作为标准查询语言。本章深入探讨关系型数据库的核心概念、技术和实现。

NOTE

关系型数据库的核心理念是用二维表组织数据，通过数学理论（关系代数）保证数据的正确性和一致性。

1 关系模型基础

关系模型由 E.F. Codd 于 1970 年提出，是现代数据库的理论基础。

1.1 基本概念

关系 (Relation)

定义：一个关系对应一张二维表。

特点：

- 每一列中的数据必须是同一类型
- 不同的列可以出自同一个域
- 列的顺序无所谓
- 行的顺序无所谓
- 任意两个元组不能完全相同

1	学生关系 (Students):				text
2					
3	学号	姓名	年龄	系别	
4					
5	001	张三	20	计算机	
6	002	李四	21	数学	
7					

元组 (Tuple)

定义：表中的一行即为一个元组。

示例：(001, '张三', 20, '计算机') 是一个元组

属性 (Attribute)

定义：表中的一列即为一个属性。

示例：学号、姓名、年龄、系别都是属性

域 (Domain)

定义：属性的取值范围。

示例：

- 学号的域：长度为 10 的字符串
- 年龄的域：正整数，范围 1-150
- 性别的域：{'男', '女'}

码 (Key)

候选码 (Candidate Key)：能唯一标识元组的最小属性集。

主码 (Primary Key)：从候选码中选择一个作为主码。

外码 (Foreign Key)：一个关系中的属性引用另一个关系的主码。

示例：

```
1 CREATE TABLE students (  
2   id VARCHAR(10) PRIMARY KEY,      -- 主码  
3   name VARCHAR(50),  
4   dept_id VARCHAR(10),  
5   FOREIGN KEY (dept_id) REFERENCES departments(id) -- 外码  
6 );
```

关系模式

定义：对关系的描述，表示为：关系名(属性1, 属性2, ..., 属性n)

示例：Students(学号, 姓名, 年龄, 系别)

TIP

关系模式是型 (Type)，关系是值 (Value)。关系模式相对稳定，关系随数据更新而变化。

1.2 关系的完整性约束

实体完整性

规则：若属性 A 是基本关系 R 的主属性，则 A 不能取空值 (NULL)。

```
1 CREATE TABLE students (  
2   id VARCHAR(10) PRIMARY KEY,      -- 自动满足实体完整性  
3   name VARCHAR(50) NOT NULL  
4 );
```

参照完整性

规则：若属性 (或属性组) F 是基本关系 R 的外码，它与基本关系 S 的主码 Ks 相对应，则对于 R 中每个元组在 F 上的值必须：

- 取空值 (F 的每个属性均为空值)

- 等于 S 中某个元组的主码值

```
1 CREATE TABLE enrollments (  
2     student_id VARCHAR(10),  
3     course_id VARCHAR(10),  
4     FOREIGN KEY (student_id) REFERENCES students(id)  
5         ON DELETE CASCADE,  
6     FOREIGN KEY (course_id) REFERENCES courses(id)  
7         ON DELETE RESTRICT  
8 );
```

用户定义的完整性

规则：针对某一具体关系数据库的约束条件。

```
1 CREATE TABLE employees (  
2     id INT PRIMARY KEY,  
3     age INT CHECK (age >= 18 AND age <= 65),  
4     gender CHAR(1) CHECK (gender IN ('M', 'F')),  
5     salary DECIMAL(10, 2) CHECK (salary > 0)  
6 );
```

2 关系代数

关系代数是关系数据库的理论基础，提供了一组操作关系的形式化方法。

2.1 基本运算

选择 (Selection, σ)

定义：从关系中选出满足给定条件的元组。

符号： $\sigma_F(R)$ ，其中 F 是选择条件

示例：

```
1  $\sigma_{\text{年龄}>20}(\text{Students})$  text  
2  
3 结果：  
4  
5 

| 学号  | 姓名 | 年龄 | 系别 |
|-----|----|----|----|
| 002 | 李四 | 21 | 数学 |

  
6  
7  
8
```

SQL 等价：

```
1 SELECT * FROM Students WHERE 年龄 > 20;
```

投影 (Projection, π)

定义：从关系中选择出若干属性列组成新的关系。

符号: $\pi_{A(R)}$, 其中 A 是属性列

示例:

1	$\pi_{\text{姓名, 系别}}(\text{Students})$		text
2			
3	结果:		
4			
5	姓名	系别	
6			
7	张三	计算机	
8	李四	数学	
9			

SQL 等价:

```
1 SELECT 姓名, 系别 FROM Students;
```

NOTE

投影会自动去除重复元组。

I 并 (Union, $\cup$)

定义: 两个相容关系的并集。

条件: 两个关系必须有相同的属性个数, 且对应属性来自相同的域。

示例:

```
1 R  $\cup$  S = {t | t  $\in$  R 或 t  $\in$  S}
```

SQL 等价:

```
1 SELECT * FROM R
2 UNION
3 SELECT * FROM S;
```

I 差 (Difference, $-$)

定义: 属于 R 但不属于 S 的元组集合。

示例:

```
1 R - S = {t | t  $\in$  R 且 t  $\notin$  S}
```

SQL 等价:

```
1 SELECT * FROM R
2 EXCEPT
3 SELECT * FROM S;
```

I 笛卡尔积 (Cartesian Product, $\times$)

定义: 两个关系的元组两两组合。

示例:

```
1 R × S
2
3 如果R有m个元组，S有n个元组，则R×S有m×n个元组
```

SQL 等价：

```
1 SELECT * FROM R, S;
2 -- 或
3 SELECT * FROM R CROSS JOIN S;
```

CAUTION

笛卡尔积的结果通常很大，实际应用中应该配合选择操作使用。

2.2 专门的关系运算

1 连接 (Join, ⋈)

定义：从两个关系的笛卡尔积中选取属性间满足一定条件的元组。

等值连接

条件为相等比较的连接。

```
1 R ⋈_(R.A=S.B) S
```

SQL 等价：

```
1 SELECT * FROM R JOIN S ON R.A = S.B;
```

自然连接 (Natural Join, ⋈)

特殊的等值连接，要求两个关系中进行比较的分量必须是相同的属性组，并且在结果中把重复的属性列去掉。

```
1 Students ⋈ Enrollments
2
3 自动在相同属性（如学号）上连接，并去除重复列
```

SQL 等价：

```
1 SELECT * FROM Students NATURAL JOIN Enrollments;
2 -- 或
3 SELECT Students.*, Enrollments.course_id, Enrollments.grade
4 FROM Students JOIN Enrollments ON Students.id = Enrollments.student_id;
```

外连接 (Outer Join)

左外连接：保留左边关系的所有元组

```
1 SELECT * FROM R LEFT JOIN S ON R.A = S.B;
```

右外连接：保留右边关系的所有元组

```
1 SELECT * FROM R RIGHT JOIN S ON R.A = S.B;
```

全外连接：保留两边关系的所有元组

```
1 SELECT * FROM R FULL OUTER JOIN S ON R.A = S.B;
```

SQL

TIP

自然连接是最常用的连接方式。外连接用于需要保留未匹配记录的场景。

I 除 (Division, $\div$)

定义：给定关系 $R(X, Y)$ 和 $S(Y, Z)$ ，其中 X, Y, Z 为属性组。 $R \div S$ 的结果是新关系 $P(X)$ ，包含所有满足以下条件的元组：该元组在 X 上的分量 x ，其象集 Y_x 包含 S 在 Y 上的投影。

通俗理解：找出与 S 中所有元组都有关联的 R 中的元组。

示例：

1 问题：查询选修了所有课程的学生

text

2

3 R (选课表): S (课程表):

学号	课程号	课程号
001	C1	C1
001	C2	C2
001	C3	C3
002	C1	
002	C2	

13

14 $R \div S = \{001\}$ (只有001选修了所有课程)

SQL 实现：

```
1 SELECT DISTINCT student_id
2 FROM enrollments e1
3 WHERE NOT EXISTS (
4     SELECT * FROM courses c
5     WHERE NOT EXISTS (
6         SELECT * FROM enrollments e2
7         WHERE e2.student_id = e1.student_id
8         AND e2.course_id = c.course_id
9     )
10 );
```

SQL

NOTE

除运算是关系代数中最复杂的运算，在实际 SQL 中通常用 NOT EXISTS 或 COUNT 来实现。

2.3 关系代数表达式的应用

示例：查询计算机系选修了“数据库”课程的学生姓名

```
1  $\pi_{\text{姓名}}(\sigma_{\text{系别}='计算机'}(\text{Students}) \bowtie \sigma_{\text{课程名}='数据库'}(\text{Courses} \bowtie \text{Enrollments}))$ 
```

text

SQL 等价：

```
1 SELECT DISTINCT s.姓名
2 FROM Students s
3 JOIN Enrollments e ON s.学号 = e.学号
4 JOIN Courses c ON e.课程号 = c.课程号
5 WHERE s.系别 = '计算机' AND c.课程名 = '数据库';
```

TIP

关系代数是 SQL 查询优化的理论基础。DBMS 会将 SQL 转换为关系代数表达式，然后进行优化。

3 SQL 语言基础

SQL (Structured Query Language) 是关系数据库的标准语言。

3.1 SQL 的特点

- 综合统一：集 DDL、DML、DCL 于一体
- 高度非过程化：只需指定“做什么”，无需说明“怎么做”
- 面向集合的操作方式
- 以同一种语法结构提供多种使用方式
- 语言简洁，易学易用

3.2 SQL 的分类

DDL (Data Definition Language)

数据定义语言，用于定义数据库结构。

创建数据库

```
1 CREATE DATABASE university
2 CHARACTER SET utf8mb4
3 COLLATE utf8mb4_unicode_ci;
```

创建表

```
1 CREATE TABLE students (
2     id VARCHAR(10) PRIMARY KEY COMMENT '学号',
3     name VARCHAR(50) NOT NULL COMMENT '姓名',
4     age INT CHECK (age >= 0 AND age <= 150) COMMENT '年龄',
5     gender ENUM('M', 'F') DEFAULT 'M' COMMENT '性别',
6     dept_id VARCHAR(10) COMMENT '系别ID',
7     email VARCHAR(100) UNIQUE COMMENT '邮箱',
8     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
9
10    FOREIGN KEY (dept_id) REFERENCES departments(id)
11    ON DELETE SET NULL
12    ON UPDATE CASCADE,
13)
```

```
14     INDEX idx_name (name),
15     INDEX idx_dept (dept_id)
16 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='学生表';
```

修改表

```
1  -- 添加列
2  ALTER TABLE students ADD COLUMN phone VARCHAR(20);
3
4  -- 修改列
5  ALTER TABLE students MODIFY COLUMN email VARCHAR(150);
6
7  -- 删除列
8  ALTER TABLE students DROP COLUMN phone;
9
10 -- 添加约束
11 ALTER TABLE students ADD CONSTRAINT uk_email UNIQUE (email);
12
13 -- 删除约束
14 ALTER TABLE students DROP INDEX uk_email;
15
16 -- 重命名表
17 ALTER TABLE students RENAME TO stu;
```

删除表

```
1 DROP TABLE IF EXISTS students;
```

CAUTION

DROP 操作不可逆，生产环境要谨慎使用。建议先备份数据。

DML (Data Manipulation Language)

数据操纵语言，用于操作数据。

插入数据

```
1  -- 插入单条记录
2  INSERT INTO students (id, name, age, dept_id)
3  VALUES ('001', '张三', 20, 'CS01');
4
5  -- 插入多条记录
6  INSERT INTO students (id, name, age, dept_id)
7  VALUES
8      ('002', '李四', 21, 'CS01'),
9      ('003', '王五', 19, 'MA01');
10
11 -- 从查询结果插入
12 INSERT INTO students_backup
13 SELECT * FROM students WHERE dept_id = 'CS01';
```

更新数据

```
1  -- 更新单条记录
2  UPDATE students
3  SET age = 21, email = 'zhangsan@example.com'
4  WHERE id = '001';
5
6  -- 批量更新
7  UPDATE students
8  SET age = age + 1
9  WHERE dept_id = 'CS01';
10
11 -- 带条件的更新
12 UPDATE students
13 SET dept_id = NULL
14 WHERE dept_id NOT IN (SELECT id FROM departments);
```

CAUTION

UPDATE 语句一定要加 WHERE 条件，否则会更新所有记录！

删除数据

```
1  -- 删除单条记录
2  DELETE FROM students WHERE id = '001';
3
4  -- 批量删除
5  DELETE FROM students WHERE age < 18;
6
7  -- 清空表（比DELETE快）
8  TRUNCATE TABLE students;
```

NOTE

TRUNCATE 是 DDL 操作，会重置自增计数器，且不能回滚。DELETE 是 DML 操作，可以回滚。

查询数据

```
1  -- 基本查询
2  SELECT id, name, age FROM students;
3
4  -- 条件查询
5  SELECT * FROM students WHERE age > 20 AND dept_id = 'CS01';
6
7  -- 排序
8  SELECT * FROM students ORDER BY age DESC, name ASC;
9
10 -- 限制结果
11 SELECT * FROM students LIMIT 10 OFFSET 20; -- 第3页，每页10条
12
13 -- 去重
14 SELECT DISTINCT dept_id FROM students;
15
```

```
16 -- 聚合
17 SELECT dept_id, COUNT(*) as count, AVG(age) as avg_age
18 FROM students
19 GROUP BY dept_id
20 HAVING COUNT(*) > 5
21 ORDER BY count DESC;
```

■ DCL (Data Control Language)

数据控制语言，用于权限管理。

```
1 -- 创建用户
2 CREATE USER 'alice'@'localhost' IDENTIFIED BY 'password123';
3
4 -- 授予权限
5 GRANT SELECT, INSERT ON university.students TO 'alice'@'localhost';
6 GRANT ALL PRIVILEGES ON university.* TO 'admin'@'%';
7
8 -- 收回权限
9 REVOKE INSERT ON university.students FROM 'alice'@'localhost';
10
11 -- 查看权限
12 SHOW GRANTS FOR 'alice'@'localhost';
13
14 -- 删除用户
15 DROP USER 'alice'@'localhost';
```

 SQL

■ TCL (Transaction Control Language)

事务控制语言，用于事务管理。

```
1 -- 开始事务
2 START TRANSACTION;
3 -- 或
4 BEGIN;
5
6 -- 提交事务
7 COMMIT;
8
9 -- 回滚事务
10 ROLLBACK;
11
12 -- 设置保存点
13 SAVEPOINT sp1;
14
15 -- 回滚到保存点
16 ROLLBACK TO sp1;
17
18 -- 设置隔离级别
19 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

 SQL

事务示例：

```
1 START TRANSACTION;
```

 SQL

```
2
3 UPDATE accounts SET balance = balance - 1000 WHERE id = 'A';
4 UPDATE accounts SET balance = balance + 1000 WHERE id = 'B';
5
6 -- 如果都成功
7 COMMIT;
8
9 -- 如果有错误
10 -- ROLLBACK;
```

TIP

始终显式管理事务，不要让 DBMS 自动提交，以保证数据一致性。

4 数据查询进阶

4.1 子查询

子查询是嵌套在其他 SQL 语句中的 SELECT 语句。

标量子查询

返回单个值的子查询。

```
1 -- 查询年龄大于平均年龄的学生
2 SELECT * FROM students
3 WHERE age > (SELECT AVG(age) FROM students);
4
5 -- 查询最高分的学生
6 SELECT * FROM students
7 WHERE id = (
8     SELECT student_id FROM grades
9     ORDER BY score DESC LIMIT 1
10 );
```

 SQL

列子查询

返回一系列值的子查询。

```
1 -- 查询计算机系和数学系的学生
2 SELECT * FROM students
3 WHERE dept_id IN (
4     SELECT id FROM departments
5     WHERE name IN ('计算机', '数学')
6 );
7
8 -- 使用ANY/SOME
9 SELECT * FROM students
10 WHERE age > ANY (
11     SELECT age FROM students WHERE dept_id = 'CS01'
12 );
```

 SQL

```
13
14 -- 使用ALL
15 SELECT * FROM students
16 WHERE age > ALL (
17     SELECT age FROM students WHERE dept_id = 'MA01'
18 );
```

行子查询

返回一行多列的子查询。

```
1 SELECT * FROM students
2 WHERE (age, dept_id) = (
3     SELECT age, dept_id FROM students WHERE id = '001'
4 );
```

表子查询

返回多行多列的子查询，通常用在 FROM 子句中。

```
1 -- 查询每个系的平均年龄
2 SELECT dept_name, avg_age
3 FROM departments d
4 JOIN (
5     SELECT dept_id, AVG(age) as avg_age
6     FROM students
7     GROUP BY dept_id
8 ) s ON d.id = s.dept_id;
```

相关子查询

子查询引用了外层查询的表。

```
1 -- 查询每个系年龄最大的学生
2 SELECT * FROM students s1
3 WHERE age = (
4     SELECT MAX(age)
5     FROM students s2
6     WHERE s2.dept_id = s1.dept_id
7 );
8
9 -- 使用EXISTS
10 SELECT * FROM departments d
11 WHERE EXISTS (
12     SELECT 1 FROM students s
13     WHERE s.dept_id = d.id
14 );
```

TIP

相关子查询对外层每一行都执行一次，性能较差。尽量用 JOIN 替代。

4.2 连接查询

内连接 (INNER JOIN)

只返回两个表中匹配的行。

```
1 SELECT s.name, d.name as dept_name
2 FROM students s
3 INNER JOIN departments d ON s.dept_id = d.id;
```

外连接 (OUTER JOIN)

左外连接

```
1 -- 查询所有学生及其系别 (包括没有系别的学生)
2 SELECT s.name, d.name as dept_name
3 FROM students s
4 LEFT JOIN departments d ON s.dept_id = d.id;
```

右外连接

```
1 -- 查询所有系别及其学生 (包括没有学生的系)
2 SELECT s.name, d.name as dept_name
3 FROM students s
4 RIGHT JOIN departments d ON s.dept_id = d.id;
```

全外连接

```
1 -- MySQL不支持FULL OUTER JOIN, 可以用UNION模拟
2 SELECT s.name, d.name as dept_name
3 FROM students s
4 LEFT JOIN departments d ON s.dept_id = d.id
5
6 UNION
7
8 SELECT s.name, d.name as dept_name
9 FROM students s
10 RIGHT JOIN departments d ON s.dept_id = d.id;
```

自连接

表与自身连接。

```
1 -- 查询员工及其上级
2 SELECT e.name as employee, m.name as manager
3 FROM employees e
4 LEFT JOIN employees m ON e.manager_id = m.id;
5
6 -- 查询同一系的学生对
7 SELECT s1.name, s2.name
8 FROM students s1
9 JOIN students s2 ON s1.dept_id = s2.dept_id
10 WHERE s1.id < s2.id; -- 避免重复
```

多表连接

```
1 -- 查询学生、课程和成绩
2 SELECT s.name, c.name as course, g.score
3 FROM students s
4 JOIN enrollments e ON s.id = e.student_id
5 JOIN courses c ON e.course_id = c.id
6 JOIN grades g ON e.id = g.enrollment_id
7 WHERE g.score >= 90;
```

NOTE

连接多个表时，注意连接顺序和索引使用，以避免性能问题。

4.3 集合运算

UNION（并集）

合并两个查询结果，去除重复。

```
1 SELECT name FROM undergraduate_students
2 UNION
3 SELECT name FROM graduate_students;
```

UNION ALL（并集，保留重复）

```
1 SELECT name FROM undergraduate_students
2 UNION ALL
3 SELECT name FROM graduate_students;
```

TIP

UNION ALL 比 UNION 快，因为不需要去重。如果确定没有重复，优先使用 UNION ALL。

INTERSECT（交集）

返回两个查询的共同结果。

```
1 -- MySQL不直接支持INTERSECT，可以用JOIN或IN实现
2 SELECT student_id FROM enrollments WHERE course_id = 'C1'
3 INTERSECT
4 SELECT student_id FROM enrollments WHERE course_id = 'C2';
5
6 -- MySQL等价写法
7 SELECT e1.student_id
8 FROM enrollments e1
9 JOIN enrollments e2 ON e1.student_id = e2.student_id
10 WHERE e1.course_id = 'C1' AND e2.course_id = 'C2';
```

EXCEPT（差集）

返回在第一个查询中但不在第二个查询中的结果。

```
1 -- MySQL不直接支持EXCEPT，可以用NOT EXISTS或LEFT JOIN实现
2 SELECT student_id FROM all_students
```

```
3  EXCEPT
4  SELECT student_id FROM graduated_students;
5
6  -- MySQL等价写法
7  SELECT student_id FROM all_students
8  WHERE student_id NOT IN (
9      SELECT student_id FROM graduated_students
10 );
```

4.4 窗口函数 (Window Functions)

窗口函数在不减少行数的情况下进行计算。

排名函数

```
1  -- ROW_NUMBER(): 连续排名
2  SELECT name, score,
3      ROW_NUMBER() OVER (ORDER BY score DESC) as rank
4  FROM students;
5
6  -- RANK(): 跳跃排名
7  SELECT name, score,
8      RANK() OVER (ORDER BY score DESC) as rank
9  FROM students;
10
11 -- DENSE_RANK(): 密集排名
12 SELECT name, score,
13     DENSE_RANK() OVER (ORDER BY score DESC) as rank
14 FROM students;
```

区别:

```
1  分数: 100, 90, 90, 80
2
3  ROW_NUMBER: 1, 2, 3, 4
4  RANK:       1, 2, 2, 4
5  DENSE_RANK: 1, 2, 2, 3
```

聚合窗口函数

```
1  -- 累计求和
2  SELECT date, amount,
3      SUM(amount) OVER (ORDER BY date) as running_total
4  FROM transactions;
5
6  -- 移动平均
7  SELECT date, price,
8      AVG(price) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) as ma7
9  FROM stock_prices;
10
11 -- 分组内排名
12 SELECT name, dept_id, salary,
```

```
13      RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) as dept_rank
14 FROM employees;
```

值函数

```
1  -- LAG(): 前一行
2  SELECT date, price,
3         LAG(price, 1) OVER (ORDER BY date) as prev_price
4  FROM stock_prices;
5
6  -- LEAD(): 后一行
7  SELECT date, price,
8         LEAD(price, 1) OVER (ORDER BY date) as next_price
9  FROM stock_prices;
10
11 -- FIRST_VALUE(): 第一个值
12 SELECT name, dept_id, salary,
13        FIRST_VALUE(salary) OVER (PARTITION BY dept_id ORDER BY hire_date) as first_salary
14 FROM employees;
```

TIP

窗口函数是 SQL 的高级特性，能解决很多复杂的分析问题。MySQL 8.0+、PostgreSQL 等都支持。

5 索引技术

索引是提高数据库查询性能的关键技术，类似于书籍的目录。

NOTE

索引的核心思想是用空间换时间：占用额外存储空间，换取查询速度的大幅提升。

5.1 索引的基本概念

为什么需要索引

没有索引：

```
1 SELECT * FROM students WHERE name = '张三';
2 -- 全表扫描，时间复杂度 O(n)
```

有索引：

```
1 CREATE INDEX idx_name ON students(name);
2 -- 索引查找，时间复杂度 O(log n)
```

性能对比：

```
1 100万条数据：
2 - 全表扫描：平均50万次比较
3 - B+树索引：约20次比较 ( $\log_2(1000000) \approx 20$ )
```

索引的代价

- 空间开销：索引占用额外存储空间
- 维护成本：INSERT、UPDATE、DELETE 时需要更新索引
- 选择成本：优化器需要选择最优索引

TIP

不是索引越多越好。频繁更新的列不适合建索引。

5.2 B+树索引

B+树是最常用的索引结构，被 MySQL InnoDB、Oracle 等广泛使用。

B+树的结构

1	[10, 20, 30]	← 根节点（内部节点）	text
2	/ \		
3	[5,10] [15,20] [25,30] [35,40]	← 内部节点	
4	/ \ / \ / \ / \		
5	[1][2][3][4][5][6][7][8][9][10]...	← 叶子节点（存储数据）	

特点：

- 所有数据都存储在叶子节点
- 叶子节点之间用链表连接（支持范围查询）
- 非叶子节点只存储键值和指针
- 树的高度通常为 2-4 层

B+树的优势

1. 磁盘 I/O 效率高：
 - 每个节点大小等于磁盘页大小（通常 16KB）
 - 一次 I/O 可以读取整个节点
 - 树高度低，I/O 次数少
2. 范围查询高效：
 - 叶子节点链表支持顺序遍历
 - `WHERE age BETWEEN 20 AND 30` 非常高效
3. 稳定性能：
 - 任何查询都需要从根到叶子的路径
 - 时间复杂度稳定为 $O(\log n)$

B+树的创建与维护

1	-- 创建索引	SQL
2	<code>CREATE INDEX idx_name ON students(name);</code>	
3		
4	-- 创建联合索引	
5	<code>CREATE INDEX idx_dept_age ON students(dept_id, age);</code>	
6		
7	-- 查看索引	

```
8  SHOW INDEX FROM students;
9
10 -- 删除索引
11 DROP INDEX idx_name ON students;
```

CAUTION

B+树索引适合等值查询和范围查询，但不适合模糊查询的前缀通配符（如 '%abc'）。

5.3 哈希索引

哈希索引基于哈希表实现，适合等值查询。

哈希索引的结构

键值	哈希函数	桶
"Alice"	————→	Bucket[3] → [记录指针]
"Bob"	————→	Bucket[7] → [记录指针]
"Carol"	————→	Bucket[3] → [记录指针] ← 哈希冲突

特点：

- 通过哈希函数直接定位
- 时间复杂度 O(1)
- 不支持范围查询
- 可能存在哈希冲突

哈希索引 vs B+树索引

特性	B+树索引	哈希索引
等值查询	O(log n)	O(1)
范围查询	支持	不支持
排序	支持	不支持
前缀匹配	支持	不支持
空间利用	较高	较低
适用场景	通用	仅等值查询

哈希索引的使用

```
1 -- MySQL Memory引擎默认使用哈希索引
2 CREATE TABLE sessions (
3     session_id VARCHAR(64) PRIMARY KEY,
4     data TEXT
5 ) ENGINE=Memory;
6
7 -- InnoDB的自适应哈希索引（自动创建）
8 -- 无法手动控制，由InnoDB根据访问模式自动决定
```

NOTE

InnoDB 引擎不支持显式创建哈希索引，但会根据访问模式自动创建自适应哈希索引。

5.4 聚簇索引与非聚簇索引

聚簇索引 (Clustered Index)

定义：数据行本身按照索引顺序存储。

特点：

- 一个表只能有一个聚簇索引
- InnoDB 的主键就是聚簇索引
- 叶子节点存储完整的数据行

1	聚簇索引结构：		text
2	[10, 20]		
3	/	\	
4	[1-10]	[11-20]	
5			
6	[完整行]	[完整行]	

非聚簇索引 (Secondary Index)

定义：索引和数据分开存储。

特点：

- 一个表可以有多个非聚簇索引
- 叶子节点存储主键值
- 需要回表查询（先查索引，再查主键索引）

1	非聚簇索引结构：		text
2	["Alice", "Bob"]		
3	/	\	
4	["Alice"]	["Bob"]	
5			
6	[PK=001]	[PK=002] ← 指向聚簇索引	

覆盖索引 (Covering Index)

定义：查询的所有列都在索引中，无需回表。

1	-- 假设创建了联合索引 idx_dept_age(dept_id, age)	SQL
2		
3	-- 覆盖索引查询（快）	
4	SELECT dept_id, age FROM students WHERE dept_id = 'CS01';	
5		
6	-- 非覆盖索引查询（慢，需要回表）	
7	SELECT dept_id, age, name FROM students WHERE dept_id = 'CS01';	

TIP

尽量设计覆盖索引，避免回表操作，可以显著提升查询性能。

5.5 索引优化策略

最左前缀原则

对于联合索引 `(a, b, c)` :

```
1 -- 可以使用索引
2 WHERE a = 1
3 WHERE a = 1 AND b = 2
4 WHERE a = 1 AND b = 2 AND c = 3
5
6 -- 不能使用索引
7 WHERE b = 2
8 WHERE c = 3
9 WHERE b = 2 AND c = 3
```

索引失效的场景

```
1 -- 1. 对索引列使用函数
2 WHERE YEAR(create_time) = 2024 -- 索引失效
3 WHERE create_time >= '2024-01-01' AND create_time < '2025-01-01' -- 索引有效
4
5 -- 2. 类型转换
6 WHERE phone = 13800138000 -- phone是VARCHAR, 索引失效
7 WHERE phone = '13800138000' -- 索引有效
8
9 -- 3. 模糊查询前缀通配符
10 WHERE name LIKE '%张' -- 索引失效
11 WHERE name LIKE '张%' -- 索引有效
12
13 -- 4. OR条件
14 WHERE id = 1 OR name = '张三' -- 如果name没有索引, 全表扫描
15
16 -- 5. NOT、!=、<>
17 WHERE age != 20 -- 索引可能失效
18
19 -- 6. IS NULL
20 WHERE age IS NULL -- 取决于数据分布
```

索引选择建议

场景	建议
高频查询列	建立索引
区分度高的列	优先索引
频繁更新的列	避免索引
小表	不需要索引
TEXT/BLOB	使用前缀索引
联合查询	考虑联合索引

前缀索引

对于长字符串, 可以只索引前缀:

```
1 -- 只索引前10个字符
2 CREATE INDEX idx_email ON students(email(10));
```

优点：节省空间 缺点：可能增加冲突

CAUTION

定期分析索引使用情况，删除未使用的索引：`SELECT * FROM sys.schema_unused_indexes;`

6 视图与存储过程

6.1 视图 (View)

视图是虚拟表，基于 SQL 查询结果集。

创建视图

```
1 -- 基本视图
2 CREATE VIEW v_computer_students AS
3 SELECT id, name, age, email
4 FROM students
5 WHERE dept_id = 'CS01';
6
7 -- 复杂视图
8 CREATE VIEW v_student_grades AS
9 SELECT
10     s.id,
11     s.name,
12     c.name AS course_name,
13     g.score,
14     RANK() OVER (PARTITION BY c.id ORDER BY g.score DESC) as rank
15 FROM students s
16 JOIN enrollments e ON s.id = e.student_id
17 JOIN courses c ON e.course_id = c.id
18 JOIN grades g ON e.id = g.enrollment_id;
```

使用视图

```
1 -- 像普通表一样查询
2 SELECT * FROM v_computer_students WHERE age > 20;
3
4 -- 连接视图
5 SELECT v.*, d.name as dept_name
6 FROM v_computer_students v
7 JOIN departments d ON v.dept_id = d.id;
```

视图的优势

1. 简化查询：封装复杂逻辑
2. 安全性：隐藏敏感字段

3. 逻辑独立性：底层表结构变化不影响应用
4. 代码复用：避免重复编写相同查询

视图的限制

- 某些视图不可更新（包含聚合、GROUP BY 等）
- 性能可能不如直接查询
- 嵌套视图可能导致性能问题

```
1  -- 可更新视图
2  CREATE VIEW v_active_students AS
3  SELECT id, name, email
4  FROM students
5  WHERE status = 'active';
6
7  UPDATE v_active_students SET email = 'new@example.com' WHERE id = '001'; -- 可以
8
9  -- 不可更新视图
10 CREATE VIEW v_avg_grades AS
11 SELECT student_id, AVG(score) as avg_score
12 FROM grades
13 GROUP BY student_id;
14
15 UPDATE v_avg_grades SET avg_score = 90; -- 错误!
```

TIP

视图不会存储数据，每次查询都会执行底层 SQL。如果需要物化结果，使用物化视图（Materialized View）。

6.2 存储过程（Stored Procedure）

存储过程是预编译的 SQL 代码块，存储在数据库中。

创建存储过程

```
1  DELIMITER //
2
3  CREATE PROCEDURE sp_transfer_money(
4      IN from_account VARCHAR(20),
5      IN to_account VARCHAR(20),
6      IN amount DECIMAL(10, 2),
7      OUT result VARCHAR(50)
8  )
9  BEGIN
10     DECLARE from_balance DECIMAL(10, 2);
11     DECLARE exit handler FOR SQLEXCEPTION
12     BEGIN
13         ROLLBACK;
14         SET result = 'TRANSFER_FAILED';
15     END;
16
```

```
17  START TRANSACTION;
18
19  -- 检查余额
20  SELECT balance INTO from_balance
21  FROM accounts
22  WHERE account_id = from_account
23  FOR UPDATE;
24
25  IF from_balance < amount THEN
26      ROLLBACK;
27      SET result = 'INSUFFICIENT_BALANCE';
28  ELSE
29      -- 扣款
30      UPDATE accounts
31      SET balance = balance - amount
32      WHERE account_id = from_account;
33
34      -- 存款
35      UPDATE accounts
36      SET balance = balance + amount
37      WHERE account_id = to_account;
38
39      COMMIT;
40      SET result = 'SUCCESS';
41  END IF;
42  END //
43
44  DELIMITER ;
```

调用存储过程

```
1  CALL sp_transfer_money('A001', 'B002', 1000.00, @result);
2  SELECT @result; -- 查看结果
```

 SQL

存储过程的优势

1. 性能：预编译，减少网络传输
2. 安全：可以控制权限
3. 封装：隐藏业务逻辑
4. 维护：集中管理业务规则

存储过程的劣势

1. 调试困难：不如应用代码易调试
2. 移植性差：不同 DBMS 语法差异大
3. 版本控制：难以纳入版本管理系统
4. 扩展性：不适合复杂业务逻辑

NOTE

现代开发趋势是将业务逻辑放在应用层，数据库层只做数据存储和简单查询。

6.3 函数 (Function)

函数与存储过程类似，但必须返回值。

```
1 DELIMITER //
2
3 CREATE FUNCTION fn_calculate_age(birth_date DATE)
4 RETURNS INT
5 DETERMINISTIC
6 BEGIN
7     RETURN TIMESTAMPDIFF(YEAR, birth_date, CURDATE());
8 END //
9
10 DELIMITER ;
11
12 -- 使用函数
13 SELECT name, fn_calculate_age(birth_date) as age
14 FROM students;
```

存储过程 vs 函数：

特性	存储过程	函数
返回值	可选 (OUT 参数)	必须
调用方式	CALL	SELECT/表达式
事务控制	支持	不支持
SQL 语句	任意	有限制
用途	业务逻辑	计算

7 触发器与事件调度

7.1 触发器 (Trigger)

触发器是在特定事件发生时自动执行的代码块。

触发器的类型

按触发时机：

- BEFORE：在操作之前执行
- AFTER：在操作之后执行

按触发事件：

- INSERT
- UPDATE
- DELETE

创建触发器

```
1 DELIMITER //
```

```
2
3  -- 审计触发器
4  CREATE TRIGGER trg_student_audit
5  AFTER INSERT ON students
6  FOR EACH ROW
7  BEGIN
8      INSERT INTO audit_log (
9          table_name, operation, record_id,
10         operation_time, operator
11     ) VALUES (
12         'students', 'INSERT', NEW.id,
13         NOW(), CURRENT_USER()
14     );
15 END //
16
17 -- 级联更新触发器
18 CREATE TRIGGER trg_update_dept_count
19 AFTER INSERT ON students
20 FOR EACH ROW
21 BEGIN
22     UPDATE departments
23     SET student_count = student_count + 1
24     WHERE id = NEW.dept_id;
25 END //
26
27 -- 数据验证触发器
28 CREATE TRIGGER trg_validate_salary
29 BEFORE INSERT ON employees
30 FOR EACH ROW
31 BEGIN
32     IF NEW.salary < 0 THEN
33         SIGNAL SQLSTATE '45000'
34         SET MESSAGE_TEXT = 'Salary cannot be negative';
35     END IF;
36 END //
37
38 DELIMITER ;
```

OLD 和 NEW 关键字

- `NEW.column`：新值（INSERT 和 UPDATE 可用）
- `OLD.column`：旧值（UPDATE 和 DELETE 可用）

```
1  CREATE TRIGGER trg_log_salary_change
2  AFTER UPDATE ON employees
3  FOR EACH ROW
4  BEGIN
5      IF OLD.salary != NEW.salary THEN
6          INSERT INTO salary_history (
7              employee_id, old_salary, new_salary, change_date
8          ) VALUES (
9              NEW.id, OLD.salary, NEW.salary, NOW()
```

 SQL

```
10      );  
11      END IF;  
12  END;
```

触发器的优势

1. 自动化：自动执行业务规则
2. 一致性：保证数据完整性
3. 审计：记录数据变更历史
4. 封装：隐藏复杂逻辑

触发器的劣势

1. 隐蔽性：不易发现，调试困难
2. 性能：可能影响批量操作性能
3. 维护：逻辑分散，难以追踪
4. 递归：可能触发连锁反应

CAUTION

谨慎使用触发器，复杂的业务逻辑应该在应用层处理。

管理触发器

```
1  -- 查看所有触发器  
2  SHOW TRIGGERS;  
3  
4  -- 查看特定表的触发器  
5  SELECT * FROM information_schema.TRIGGERS  
6  WHERE EVENT_OBJECT_TABLE = 'students';  
7  
8  -- 删除触发器  
9  DROP TRIGGER IF EXISTS trg_student_audit;
```

[SQL](#)

7.2 事件调度器 (Event Scheduler)

事件调度器用于定时执行任务，类似于 cron job。

启用事件调度器

```
1  -- 查看状态  
2  SHOW VARIABLES LIKE 'event_scheduler';  
3  
4  -- 启用  
5  SET GLOBAL event_scheduler = ON;  
6  
7  -- 永久启用 (my.cnf)  
8  -- [mysqld]  
9  -- event_scheduler=ON
```

[SQL](#)

创建事件

```
1  -- 每天凌晨清理过期数据
2  CREATE EVENT evt_cleanup_expired_data
3  ON SCHEDULE EVERY 1 DAY
4  STARTS '2024-01-01 00:00:00'
5  DO
6      DELETE FROM sessions
7      WHERE expire_time < NOW();
8
9  -- 每小时统计一次
10 CREATE EVENT evt_hourly_stats
11 ON SCHEDULE EVERY 1 HOUR
12 DO
13     INSERT INTO hourly_stats (
14         stat_time, total_users, active_users
15     )
16     SELECT
17         NOW(),
18         COUNT(*),
19         SUM(CASE WHEN last_login > NOW() - INTERVAL 1 HOUR THEN 1 ELSE 0 END)
20     FROM users;
21
22 -- 一次性事件
23 CREATE EVENT evt_cleanup_once
24 ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 1 HOUR
25 DO
26     CALL sp_cleanup_temp_tables();
```

事件调度选项

```
1  -- 指定结束时间
2  ON SCHEDULE EVERY 1 DAY
3  ENDS '2024-12-31 23:59:59'
4
5  -- 禁用/启用
6  ALTER EVENT evt_cleanup_expired_data DISABLE;
7  ALTER EVENT evt_cleanup_expired_data ENABLE;
8
9  -- 修改调度
10 ALTER EVENT evt_cleanup_expired_data
11 ON SCHEDULE EVERY 12 HOUR;
12
13 -- 删除事件
14 DROP EVENT IF EXISTS evt_cleanup_expired_data;
```

查看事件

```
1  -- 查看所有事件
2  SHOW EVENTS;
3
4  -- 查看详细信息
5  SELECT * FROM information_schema.EVENTS
```


第二范式 (2NF)

定义：满足 1NF，且每个非主属性完全函数依赖于码（不存在部分依赖）。

问题示例：

1	选课(学号, 课程号, 成绩, 课程名, 学分)	text
2	码: (学号, 课程号)	
3		
4	部分依赖:	
5	(学号, 课程号) → 成绩 ✓ 完全依赖	
6	课程号 → 课程名 X 部分依赖	
7	课程号 → 学分 X 部分依赖	

分解为 2NF：

1	选课(学号, 课程号, 成绩)	text
2	课程(课程号, 课程名, 学分)	

TIP

2NF 主要解决复合主键的部分依赖问题。如果主键是单属性，自动满足 2NF。

第三范式 (3NF)

定义：满足 2NF，且每个非主属性不传递依赖于码。

问题示例：

1	学生(学号, 姓名, 系别, 系主任)	text
2	码: 学号	
3		
4	传递依赖:	
5	学号 → 系别 → 系主任	

分解为 3NF：

1	学生(学号, 姓名, 系别)	text
2	系别(系别, 系主任)	

BC 范式 (BCNF)

定义：满足 1NF，且对于每一个函数依赖 $X \rightarrow Y$ ($Y \not\subset X$)， X 都包含码。

通俗理解：每个决定因素都包含码。

问题示例：

1	选课(学生, 教师, 课程)	text
2		
3	函数依赖:	
4	(学生, 课程) → 教师	
5	教师 → 课程	
6		
7	码: (学生, 课程) 和 (学生, 教师)	
8		

9 问题：教师 → 课程，但教师不包含码

分解为 BCNF：

- 1 教学(学生, 教师)
- 2 授课(教师, 课程)

NOTE

BCNF 比 3NF 更严格。大多数情况下，3NF 已经足够好。

8.3 范式对比

范式	要求	解决的问题
1NF	属性原子性	数据不可分
2NF	消除部分依赖	数据冗余、插入异常
3NF	消除传递依赖	数据冗余、更新异常
BCNF	决定因素含码	剩余的数据冗余

8.4 规范化示例

原始表：

- 1 订单(订单号, 日期, 客户号, 客户名, 客户地址,
- 2 产品号, 产品名, 单价, 数量, 金额)

分析问题：

- 客户信息重复（违反 3NF）
- 产品信息重复（违反 3NF）
- 金额可以计算得出（派生属性）

规范化后：

```
1  -- 客户表
2  CREATE TABLE customers (
3      customer_id VARCHAR(10) PRIMARY KEY,
4      name VARCHAR(50),
5      address VARCHAR(200)
6  );
7
8  -- 产品表
9  CREATE TABLE products (
10     product_id VARCHAR(10) PRIMARY KEY,
11     name VARCHAR(100),
12     unit_price DECIMAL(10, 2)
13 );
14
15 -- 订单表
16 CREATE TABLE orders (
17     order_id VARCHAR(20) PRIMARY KEY,
```

```
18     order_date DATE,
19     customer_id VARCHAR(10),
20     FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
21 );
22
23 -- 订单明细表
24 CREATE TABLE order_items (
25     order_id VARCHAR(20),
26     product_id VARCHAR(10),
27     quantity INT,
28     PRIMARY KEY (order_id, product_id),
29     FOREIGN KEY (order_id) REFERENCES orders(order_id),
30     FOREIGN KEY (product_id) REFERENCES products(product_id)
31 );
```

CAUTION

规范化程度越高，表越多，查询时需要更多的 JOIN 操作，可能影响性能。

9 反规范化设计与性能优化

反规范化是有意引入冗余以提高查询性能的设计策略。

9.1 为什么要反规范化

规范化的缺点：

- 表数量多，JOIN 操作复杂
- 查询性能可能较差
- 需要更多的 I/O 操作

反规范化的优势：

- 减少 JOIN，提高查询速度
- 简化查询逻辑
- 适合读多写少的场景

TIP

反规范化是一种权衡：用存储空间和写入性能换取读取性能。

9.2 常见的反规范化技术

冗余字段

```
1 -- 规范化设计
2 SELECT o.order_id, c.name, c.address
3 FROM orders o
4 JOIN customers c ON o.customer_id = c.customer_id;
5
6 -- 反规范化：在订单表中冗余客户信息
7 ALTER TABLE orders
```

 SQL

```
8  ADD COLUMN customer_name VARCHAR(50),
9  ADD COLUMN customer_address VARCHAR(200);
10
11 -- 查询简化
12 SELECT order_id, customer_name, customer_address
13 FROM orders
14 WHERE order_id = 'ORD001';
```

维护策略：

```
1  -- 使用触发器保持数据一致
2  CREATE TRIGGER trg_sync_customer_info
3  AFTER UPDATE ON customers
4  FOR EACH ROW
5  BEGIN
6      UPDATE orders
7      SET customer_name = NEW.name,
8          customer_address = NEW.address
9      WHERE customer_id = NEW.customer_id;
10 END;
```

 SQL

I 汇总表

```
1  -- 创建汇总表
2  CREATE TABLE daily_sales_summary (
3      sale_date DATE PRIMARY KEY,
4      total_orders INT,
5      total_amount DECIMAL(12, 2),
6      avg_order_amount DECIMAL(10, 2)
7  );
8
9  -- 定时更新（使用事件调度器）
10 CREATE EVENT evt_update_daily_summary
11 ON SCHEDULE EVERY 1 DAY
12 DO
13     INSERT INTO daily_sales_summary (
14         sale_date, total_orders, total_amount, avg_order_amount
15     )
16     SELECT
17         DATE(order_date),
18         COUNT(*),
19         SUM(amount),
20         AVG(amount)
21     FROM orders
22     WHERE DATE(order_date) = CURDATE() - INTERVAL 1 DAY
23     ON DUPLICATE KEY UPDATE
24         total_orders = VALUES(total_orders),
25         total_amount = VALUES(total_amount),
26         avg_order_amount = VALUES(avg_order_amount);
```

 SQL

I 物化视图

```
1  -- PostgreSQL物化视图
```

 SQL

```
2 CREATE MATERIALIZED VIEW mv_monthly_sales AS
3 SELECT
4     DATE_TRUNC('month', order_date) as month,
5     COUNT(*) as order_count,
6     SUM(amount) as total_sales
7 FROM orders
8 GROUP BY DATE_TRUNC('month', order_date);
9
10 -- 刷新物化视图
11 REFRESH MATERIALIZED VIEW mv_monthly_sales;
12
13 -- MySQL可以使用表模拟
14 CREATE TABLE mv_monthly_sales AS
15 SELECT ...
```

宽表设计

将多个相关表合并为一个大宽表：

```
1 CREATE TABLE order_wide_table (
2     order_id VARCHAR(20) PRIMARY KEY,
3     order_date DATE,
4     customer_id VARCHAR(10),
5     customer_name VARCHAR(50),      -- 冗余
6     customer_email VARCHAR(100),    -- 冗余
7     product_id VARCHAR(10),
8     product_name VARCHAR(100),      -- 冗余
9     product_category VARCHAR(50),   -- 冗余
10    quantity INT,
11    unit_price DECIMAL(10, 2),
12    total_amount DECIMAL(12, 2)
13 );
```

适用场景：

- 数据仓库
- OLAP 分析
- 报表系统

NOTE

宽表设计在 OLTP 系统中应谨慎使用，更适合 OLAP 场景。

9.3 反规范化的原则

何时使用反规范化

场景	建议
读多写少	适合反规范化
实时性要求高	适合反规范化
复杂 JOIN 频繁	考虑反规范化
写多读少	保持规范化

数据一致性要求高	谨慎反规范化
存储空间紧张	保持规范化

最佳实践

- 1. 先规范化，后反规范化：
 - 先设计规范的数据库
 - 根据性能瓶颈进行反规范化
- 2. 文档化：
 - 记录所有冗余字段
 - 说明维护策略
- 3. 自动化维护：
 - 使用触发器
 - 使用定时任务
 - 使用应用层逻辑
- 4. 监控一致性：
 - 定期检查数据一致性
 - 提供数据修复工具
- 5. 权衡利弊：
 - 评估性能提升
 - 评估维护成本
 - 评估一致性风险

9.4 性能优化综合策略

查询优化

SQL

```
1  -- 1. 避免SELECT *
2  SELECT id, name FROM students; -- 好
3  SELECT * FROM students;      -- 不好
4
5  -- 2. 使用EXISTS代替IN
6  SELECT * FROM departments d
7  WHERE EXISTS (
8      SELECT 1 FROM students s WHERE s.dept_id = d.id
9  );
10
11 -- 3. 避免在索引列上使用函数
12 WHERE create_time >= '2024-01-01' -- 好
13 WHERE YEAR(create_time) = 2024    -- 不好
14
15 -- 4. 使用LIMIT限制结果集
16 SELECT * FROM students LIMIT 100;
17
18 -- 5. 分析查询计划
19 EXPLAIN SELECT * FROM students WHERE dept_id = 'CS01';
```

配置优化

```
1  -- MySQL配置优化示例
2
3  -- 缓冲池大小（建议为物理内存的50-70%）
4  SET GLOBAL innodb_buffer_pool_size = 4294967296; -- 4GB
5
6  -- 连接数
7  SET GLOBAL max_connections = 500;
8
9  -- 查询缓存（MySQL 8.0已移除）
10 -- query_cache_size = 64M
11
12 -- 日志配置
13 SET GLOBAL slow_query_log = 'ON';
14 SET GLOBAL long_query_time = 2; -- 超过2秒的记录
```

硬件优化

- 使用 SSD 替代 HDD
- 增加内存（提高缓存命中率）
- RAID 配置（RAID 10 适合数据库）
- 专用数据库服务器

架构优化

- 读写分离
- 分库分表
- 缓存层（Redis、Memcached）
- CDN 加速静态资源

10 锁机制与并发控制

锁是数据库实现并发控制的核心机制，用于保证多事务并发执行时的数据一致性。

NOTE

锁的本质是协调多个进程/线程对共享资源的访问，在保证一致性的同时尽可能提高并发性。

10.1 锁的类型

共享锁（S 锁，读锁）

定义：允许事务读取数据，其他事务也可以获得共享锁，但不能获得排他锁。

特点：

- 多个事务可以同时持有 S 锁
- 持有 S 锁期间，其他事务不能获取 X 锁
- 用于读操作

```
1  -- MySQL中显式加共享锁
```

```
2 SELECT * FROM students WHERE id = '001' LOCK IN SHARE MODE;
```

排他锁（X 锁，写锁）

定义：允许事务修改数据，其他事务不能获得任何类型的锁。

特点：

- 只有一个事务可以持有 X 锁
- 持有 X 锁期间，其他事务不能获取 S 锁或 X 锁
- 用于写操作（INSERT、UPDATE、DELETE）

```
1 -- MySQL中显式加排他锁
2 SELECT * FROM students WHERE id = '001' FOR UPDATE;
```

SQL

意向锁（Intention Lock）

定义：表级锁，表示事务打算在行上加什么类型的锁。

类型：

- 意向共享锁（IS）：事务打算在行上加 S 锁
- 意向排他锁（IX）：事务打算在行上加 X 锁

作用：

- 快速判断表是否可以加表锁
- 避免遍历所有行检查锁状态

```
1 示例：
2 事务T1要在某行加X锁：
3 1. 先在表上加IX锁
4 2. 再在行上加X锁
5
6 事务T2要对整个表加X锁：
7 - 检查表上有IX锁 → 冲突 → 等待
```

text

TIP

意向锁是 InnoDB 自动添加的，用户无法手动控制。

10.2 锁的粒度

锁粒度	范围	并发性	开销	示例
全局锁	整个数据库	低	小	FTWRL
表级锁	整张表	中	小	LOCK TABLES
页级锁	数据页	较高	中	BDB 引擎
行级锁	单行记录	高	大	InnoDB 默认

行级锁（Row-Level Lock）

特点：

- 粒度最小，并发性最高

- 开销最大
- InnoDB 默认使用

实现：通过索引项加锁

```
1 -- 如果id有索引，只锁定该行
2 SELECT * FROM students WHERE id = '001' FOR UPDATE;
3
4 -- 如果id没有索引，会锁定全表！（退化为表锁）
5 SELECT * FROM students WHERE name = '张三' FOR UPDATE;
```

CAUTION

行锁是通过索引实现的，如果查询条件没有用到索引，会升级为表锁！

表级锁（Table-Level Lock）

特点：

- 粒度大，并发性低
- 开销小
- MyISAM 引擎使用

```
1 -- 显式加表锁
2 LOCK TABLES students READ;
3 LOCK TABLES students WRITE;
4
5 -- 释放锁
6 UNLOCK TABLES;
```

页级锁（Page-Level Lock）

特点：

- 介于行锁和表锁之间
- Berkeley DB 引擎使用
- MySQL 中较少见

10.3 InnoDB 的锁算法

记录锁（Record Lock）

定义：锁定索引记录本身。

```
1 -- 锁定id='001'这一行
2 SELECT * FROM students WHERE id = '001' FOR UPDATE;
```

间隙锁（Gap Lock）

定义：锁定索引记录之间的间隙，防止其他事务插入新记录。

示例：

```
1 表中有记录：id = 1, 3, 5, 7
2
3 事务T1执行：
```

```

4 SELECT * FROM students WHERE id = 4 FOR UPDATE;
5
6 间隙锁锁定: (3, 5) 这个区间
7
8 事务T2尝试:
9 INSERT INTO students VALUES (4, ...); -- 被阻塞!

```

作用：防止幻读

NOTE

间隙锁只在可重复读（REPEATABLE READ）隔离级别下生效。

临键锁（Next-Key Lock）

定义：记录锁 + 间隙锁的组合，锁定一个范围并包括记录本身。

示例：

```

1 表中有记录: id = 1, 3, 5, 7
2
3 事务T1执行:
4 SELECT * FROM students WHERE id = 3 FOR UPDATE;
5
6 临键锁锁定: (1, 3] 这个区间（不包括1, 包括3）

```

锁定规则：

```

1 对于索引值: 1, 3, 5, 7
2
3 可能的临键锁区间:
4 (-∞, 1]
5 (1, 3]
6 (3, 5]
7 (5, 7]
8 (7, +∞)

```

TIP

临键锁是 InnoDB 在 RR 级别下防止幻读的关键机制。

10.4 死锁（Deadlock）

死锁的产生

当两个或多个事务互相等待对方释放锁时，形成循环等待。

时间	事务T1	事务T2
t1	LOCK X on A	
t2		LOCK X on B
t3	LOCK X on B ← 等待T2释放	
t4		LOCK X on A ← 等待T1释放
结果	死锁!	

死锁的必要条件

1. 互斥条件：资源一次只能被一个事务占用
2. 请求与保持：事务持有资源的同时请求新资源
3. 不可剥夺：资源不能被强制剥夺
4. 循环等待：存在循环等待链

死锁的检测与处理

InnoDB 的死锁检测：

- 维护等待图（Wait-For Graph）
- 定期检测图中是否有环
- 发现死锁后，回滚代价最小的事务

```
1 -- 查看死锁信息
2 SHOW ENGINE INNODB STATUS\G
3
4 -- 输出中包含：
5 -- LATEST DETECTED DEADLOCK
6 -- 显示死锁的详细信息
```

配置参数：

```
1 -- 死锁检测超时时间（秒）
2 SET GLOBAL innodb_lock_wait_timeout = 50;
3
4 -- 是否启用死锁检测
5 SET GLOBAL innodb_deadlock_detect = ON;
```

死锁的预防

1. 固定顺序加锁：

```
1 -- 所有事务都按相同顺序访问资源
2 -- T1: LOCK A, then LOCK B
3 -- T2: LOCK A, then LOCK B （不是先B后A）
```

2. 一次性加锁：

- 事务开始时获取所有需要的锁
- 缺点：资源利用率低

3. 设置超时：

```
1 SET innodb_lock_wait_timeout = 10;
2 -- 超过10秒未获取锁，自动回滚
```

4. 使用较低的隔离级别：

- READ COMMITTED 不使用间隙锁
- 减少死锁概率

CAUTION

死锁是正常现象，应用层应该有重试机制。

死锁的处理策略

应用层重试：

```
1 import time
2 import MySQLdb
3
4 def execute_with_retry(sql, max_retries=3):
5     for attempt in range(max_retries):
6         try:
7             cursor.execute(sql)
8             connection.commit()
9             return
10        except MySQLdb.OperationalError as e:
11            if e.args[0] == 1213: # 死锁错误码
12                connection.rollback()
13                time.sleep(0.1 * (2 ** attempt)) # 指数退避
14                continue
15            raise
16        raise Exception("Max retries exceeded")
```

11 事务隔离级别

事务隔离级别决定了事务之间的可见性和并发程度。

11.1 SQL 标准定义的隔离级别

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED	可能	可能	可能
READ COMMITTED	不可能	可能	可能
REPEATABLE READ	不可能	不可能	可能
SERIALIZABLE	不可能	不可能	不可能

注：InnoDB 在 RR 级别下通过 MVCC 和临键锁解决了幻读问题。

11.2 各隔离级别详解

读未提交（READ UNCOMMITTED）

特点：

- 最低隔离级别
- 可以读取其他事务未提交的数据
- 并发性最高，但安全性最低

示例：

```
1 -- 事务T1
2 START TRANSACTION;
```

```
3  UPDATE accounts SET balance = balance - 100 WHERE id = 'A';
4  -- 尚未COMMIT
5
6  -- 事务T2 (READ UNCOMMITTED级别)
7  SELECT balance FROM accounts WHERE id = 'A';
8  -- 读到的是T1未提交的值 (脏读)
9
10 -- T1回滚
11 ROLLBACK;
12
13 -- T2读到的数据是错误的!
```

使用场景：几乎不使用，除非对数据准确性要求极低

I 读已提交 (READ COMMITTED, RC)

特点：

- Oracle、SQL Server 的默认级别
- 只能读取已提交的数据
- 解决脏读，但存在不可重复读

示例：

```
1  -- 事务T1
2  START TRANSACTION;
3  SELECT balance FROM accounts WHERE id = 'A'; -- 余额1000
4
5  -- 事务T2
6  START TRANSACTION;
7  UPDATE accounts SET balance = 900 WHERE id = 'A';
8  COMMIT;
9
10 -- 事务T1再次查询
11 SELECT balance FROM accounts WHERE id = 'A'; -- 余额900 (不可重复读)
```

InnoDB 的实现：

- 每次 SELECT 都创建新的 ReadView
- 看到最新已提交的数据

优点：

- 减少锁竞争
- 适合写多读少的场景

TIP

RC 级别下，InnoDB 使用快照读 (Snapshot Read)，不加锁。

I 可重复读 (REPEATABLE READ, RR)

特点：

- MySQL InnoDB 的默认级别
- 同一事务中多次读取结果一致
- 解决脏读和不可重复读

示例:

```

1  -- 事务T1
2  START TRANSACTION;
3  SELECT balance FROM accounts WHERE id = 'A'; -- 余额1000
4
5  -- 事务T2
6  START TRANSACTION;
7  UPDATE accounts SET balance = 900 WHERE id = 'A';
8  COMMIT;
9
10 -- 事务T1再次查询
11 SELECT balance FROM accounts WHERE id = 'A'; -- 仍然是1000 (可重复读)

```

InnoDB 的实现:

- 事务开始时创建 ReadView
- 整个事务期间使用同一个 ReadView
- 配合 MVCC 实现一致性读

幻读问题:

```

1  -- 事务T1
2  START TRANSACTION;
3  SELECT * FROM students WHERE age > 20; -- 返回2条
4
5  -- 事务T2
6  INSERT INTO students VALUES ('003', '王五', 25);
7  COMMIT;
8
9  -- 事务T1再次查询
10 SELECT * FROM students WHERE age > 20; -- 仍然返回2条 (RR级别)
11
12 -- 但如果T1尝试更新
13 UPDATE students SET age = 21 WHERE age > 20; -- 会影响3条 (包括T2插入的)

```

NOTE

InnoDB 通过临键锁 (Next-Key Lock) 在 RR 级别下防止了大部分幻读场景。

I 串行化 (SERIALIZABLE)

特点:

- 最高隔离级别
- 事务串行执行
- 解决所有并发问题
- 性能最差

实现:

- 所有 SELECT 自动加共享锁
- 相当于单线程执行

```

1 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
2

```

```
3 START TRANSACTION;
4 SELECT * FROM students WHERE id = '001'; -- 自动加S锁
5 -- 其他事务不能修改这行，直到本事务结束
```

使用场景：

- 对数据一致性要求极高
- 并发量低的场景
- 金融系统等关键业务

CAUTION

SERIALIZABLE 级别会严重降低并发性能，慎用！

11.3 隔离级别的实现机制

基于锁的实现

- SERIALIZABLE：完全基于锁
- RR：锁 + MVCC
- RC：MVCC 为主
- RU：几乎不加锁

基于 MVCC 的实现

- RC 和 RR 主要依赖 MVCC
- 通过 ReadView 控制可见性
- 减少锁的使用，提高并发

11.4 选择合适的隔离级别

场景	推荐级别	原因
一般业务	RR	MySQL 默认，平衡性能和一致性
高并发写入	RC	减少锁竞争
数据分析	RC	需要最新数据
金融交易	RR 或 SERIALIZABLE	强一致性要求
报表统计	RC	容忍轻微不一致

TIP

大多数应用使用默认的 RR 级别即可。如果有特殊需求，再考虑调整。

12 MVCC 原理

MVCC（Multi-Version Concurrency Control，多版本并发控制）是实现高并发的重要技术。

NOTE

MVCC 的核心思想是保存数据的历史版本，通过版本控制实现非阻塞读。

12.1 MVCC 的基本概念

什么是 MVCC

MVCC 允许读写操作不互相阻塞：

- 读操作读取历史版本（快照读）
- 写操作创建新版本
- 读写互不干扰

对比传统锁机制：

1	传统锁：	text
2	读 → 加S锁 → 读取 → 释放锁	
3	写 → 加X锁 → 修改 → 释放锁	
4	读和写互相阻塞	
5		
6	MVCC：	
7	读 → 读取快照（不加锁）	
8	写 → 创建新版本	
9	读和写不阻塞	

InnoDB 中的 MVCC 实现

InnoDB 通过以下机制实现 MVCC：

1. 隐藏字段：每行记录包含隐藏列
2. Undo Log：保存历史版本
3. ReadView：控制版本可见性

12.2 InnoDB 的行记录结构

隐藏字段

InnoDB 的每行记录包含三个隐藏字段：

字段	说明	用途
DB_TRX_ID	最近修改事务 ID	判断版本可见性
DB_ROLL_PTR	回滚指针	指向 Undo Log
DB_ROW_ID	隐藏行 ID	生成聚簇索引

1 行记录结构:

2				
3	DB_TRX_ID	DB_ROLL_PTR	DB_ROW_ID	实际数据
4	(6字节)	(7字节)	(6字节)	(变长)
5				

12.3 Undo Log 与版本链

Undo Log

定义：记录数据修改前的值，用于回滚和 MVCC。

类型:

- Insert Undo Log: INSERT 操作产生, 事务提交后可删除
- Update Undo Log: UPDATE/DELETE 操作产生, 需要保留供 MVCC 使用

版本链

多次修改同一行记录时, 通过回滚指针形成版本链:

1	当前版本 (TRX_ID=103)	text
2	↑	
3	DB_ROLL_PTR	
4	↑	
5	历史版本1 (TRX_ID=102)	
6	↑	
7	DB_ROLL_PTR	
8	↑	
9	历史版本2 (TRX_ID=101)	
10	↑	
11	DB_ROLL_PTR	
12	↑	
13	初始版本 (TRX_ID=100)	

示例:

1	-- 初始状态: balance = 1000 (TRX_ID=100)	SQL
2		
3	-- 事务T1 (TRX_ID=101)	
4	UPDATE accounts SET balance = 900 WHERE id = 'A';	
5	-- 创建新版本: balance = 900 (TRX_ID=101)	
6	-- Undo Log记录: balance = 1000	
7		
8	-- 事务T2 (TRX_ID=102)	
9	UPDATE accounts SET balance = 800 WHERE id = 'A';	
10	-- 创建新版本: balance = 800 (TRX_ID=102)	
11	-- Undo Log记录: balance = 900	
12		
13	-- 版本链:	
14	-- 800 (TRX=102) → 900 (TRX=101) → 1000 (TRX=100)	

12.4 ReadView (读视图)

ReadView 是 MVCC 的核心, 决定事务能看到哪个版本的数据。

ReadView 的结构

1	ReadView包含:	text
2	- m_ids: 活跃事务ID列表 (启动ReadView时还未提交的事务)	
3	- min_trx_id: m_ids中的最小值	
4	- max_trx_id: 下一个要分配的事务ID	
5	- creator_trx_id: 创建ReadView的事务ID	

可见性判断算法

对于某个版本的数据 (TRX_ID = trx_id):

1	1. 如果 <code>trx_id < min_trx_id</code>	text
2	→ 版本已提交, 可见	
3		
4	2. 如果 <code>trx_id >= max_trx_id</code>	
5	→ 版本在ReadView之后生成, 不可见	
6		
7	3. 如果 <code>min_trx_id <= trx_id < max_trx_id</code>	
8	a. 如果 <code>trx_id</code> 在 <code>m_ids</code> 中	
9	→ 事务仍活跃, 不可见	
10	b. 如果 <code>trx_id</code> 不在 <code>m_ids</code> 中	
11	→ 事务已提交, 可见	

图示:

1	事务ID轴:	text
2	← 可见 ——— 不可见 ——— 未来 →	
3	min_trx_id max_trx_id	
4		
5	<code>m_ids</code> 中的事务ID: 不可见	
6	<code>m_ids</code> 外且 <code><max</code> 的: 可见	

RC 和 RR 的 ReadView 差异

READ COMMITTED

- 每次 SELECT 都创建新的 ReadView
- 能看到最新已提交的数据

1	T1开始 (TRX=100)	text
2	SELECT → 创建ReadView1 (<code>m_ids=[100]</code>)	
3		
4	T2提交 (TRX=101)	
5		
6	T1再次SELECT → 创建ReadView2 (<code>m_ids=[100]</code>)	
7	→ 能看到T2的修改	

REPEATABLE READ

- 第一次 SELECT 时创建 ReadView
- 整个事务期间复用同一个 ReadView

1	T1开始 (TRX=100)	text
2	SELECT → 创建ReadView (<code>m_ids=[100]</code>)	
3		
4	T2提交 (TRX=101)	
5		
6	T1再次SELECT → 复用ReadView (<code>m_ids=[100]</code>)	
7	→ 看不到T2的修改 (保持一致性)	

TIP

这就是 RC 和 RR 级别下行为差异的根本原因！

12.5 MVCC 的操作类型

快照读 (Snapshot Read)

定义：读取数据的快照版本，不加锁。

特点：

- 使用 MVCC
- 非阻塞
- SELECT 语句（不加锁）

```
1 -- 快照读
2 SELECT * FROM students WHERE id = '001';
3 SELECT * FROM students WHERE age > 20;
```

SQL

当前读 (Current Read)

定义：读取最新版本的数据，加锁。

特点：

- 不使用 MVCC
- 阻塞其他事务
- 需要加锁

```
1 -- 当前读
2 SELECT * FROM students WHERE id = '001' LOCK IN SHARE MODE; -- S锁
3 SELECT * FROM students WHERE id = '001' FOR UPDATE; -- X锁
4 UPDATE students SET age = 21 WHERE id = '001'; -- X锁
5 DELETE FROM students WHERE id = '001'; -- X锁
6 INSERT INTO students VALUES (...); -- X锁
```

SQL

NOTE

理解快照读和当前读的区别，是掌握 MVCC 的关键。

12.6 MVCC 的优势与劣势

优势

1. 高并发：读写不阻塞
2. 无锁读：提高读取性能
3. 一致性：提供一致性视图
4. 非阻塞：减少等待时间

劣势

1. 存储开销：需要保存多个版本
2. 清理开销：需要 Purge 线程清理旧版本

3. 复杂性：实现复杂，调试困难
4. 长事务问题：阻止旧版本清理

CAUTION

避免长时间运行的事务，它们会阻止 Undo Log 的清理，导致存储空间增长。

12.7 MVCC 的清理机制

Purge 线程

InnoDB 后台线程负责清理不再需要的版本：

```
1 清理条件:
2  1. 事务已提交
3  2. 所有活跃的ReadView都不需要该版本
4
5 清理过程:
6  1. 遍历Undo Log
7  2. 判断版本是否可见
8  3. 删除不可见的版本
9  4. 释放空间
```

配置参数

```
1  -- Purge线程数量
2  SET GLOBAL innodb_purge_threads = 4;
3
4  -- Purge批次大小
5  SET GLOBAL innodb_purge_batch_size = 300;
```

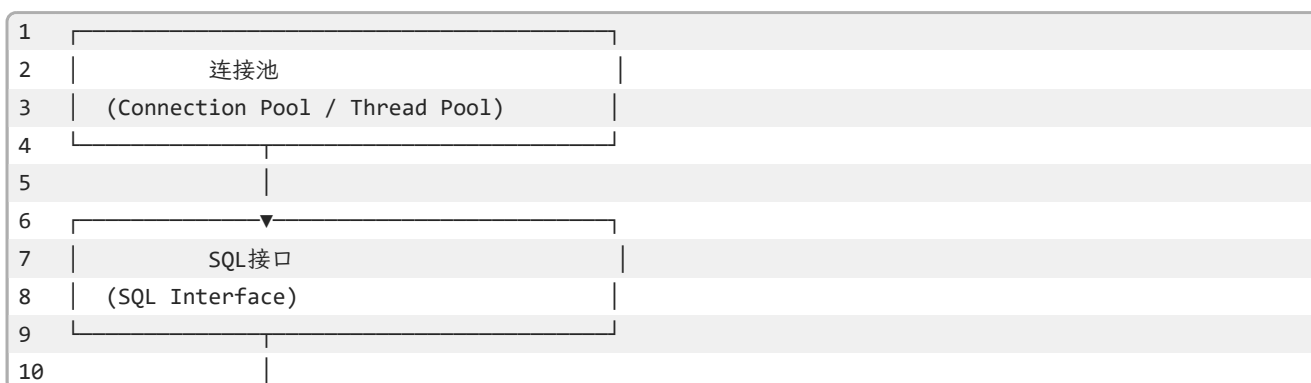
13 MySQL 架构与存储引擎

MySQL 是最流行的开源关系型数据库，采用插件式存储引擎架构。

NOTE

MySQL 的核心设计理念是模块化和可扩展性，通过插件式存储引擎实现不同场景的优化。

13.1 MySQL 整体架构





各层职责

1. 连接层：

- 处理客户端连接
- 身份验证
- 线程管理

2. SQL 接口层：

- 接收 SQL 语句
- 返回查询结果

3. 解析器：

- 词法分析
- 语法分析
- 生成解析树

4. 优化器：

- 选择最优执行计划
- 索引选择
- JOIN 顺序优化

5. 缓存层：

- Query Cache (MySQL 8.0 已移除)
- Buffer Pool (InnoDB 缓冲池)

6. 存储引擎层:

- 数据存储和提取
- 事务支持
- 锁机制

TIP

MySQL 的架构设计使得可以轻松添加新的存储引擎，而不需要修改上层代码。

13.2 InnoDB 存储引擎

InnoDB 是 MySQL 的默认存储引擎，支持事务和外键。

核心特性

特性	InnoDB	说明
事务支持	✓	ACID 兼容
行级锁	✓	高并发
外键	✓	参照完整性
MVCC	✓	非阻塞读
崩溃恢复	✓	Redo Log + Undo Log
聚簇索引	✓	主键组织数据
热备份	✓	在线备份

内存结构

Buffer Pool

作用：缓存数据和索引页

```
1 -- 配置Buffer Pool大小（建议为物理内存的50-70%）
2 SET GLOBAL innodb_buffer_pool_size = 4G;
3
4 -- 查看命中率
5 SHOW STATUS LIKE 'Innodb_buffer_pool_read%';
6
7 -- 命中率计算:
8 -- (1 - Innodb_buffer_pool_reads / Innodb_buffer_pool_read_requests) * 100%
9 -- 应该 > 99%
```

组成:

- 数据页缓存
- 索引页缓存
- 插入缓冲 (Insert Buffer)
- 自适应哈希索引 (Adaptive Hash Index)
- 锁信息

Redo Log (重做日志)

作用：保证事务的持久性

特点：

- 物理日志（记录“在某个数据页上做了什么修改”）
- 循环写入
- Crash-Safe 能力

1	Redo Log结构:				text
2					
3	write	write	write	write	
4	pos	pos	pos	pos	
5					
6	↑				
7	checkpoint				

工作流程：

1	1. 事务修改数据
2	2. 先写Redo Log (prepare状态)
3	3. 再写Binlog
4	4. 提交事务 (commit状态)
5	
6	如果崩溃：
7	- Redo Log中有记录 → 恢复数据
8	- Redo Log中无记录 → 回滚事务

Undo Log（回滚日志）

作用：

- 事务回滚
- MVCC 版本控制

特点：

- 逻辑日志（记录相反操作）
- INSERT 的 Undo Log 可在事务提交后删除
- UPDATE/DELETE 的 Undo Log 需要保留供 MVCC 使用

磁盘结构

表空间（Tablespace）

系统表空间：

```
1 ibdata1 # 默认系统表空间
```

Bash

独立表空间（推荐）：

```
1 table_name.ibd # 每个表一个文件
```

Bash

```
1 -- 启用独立表空间（默认开启）
2 SET GLOBAL innodb_file_per_table = ON;
```

SQL

优势：

- 便于管理
- 可以快速回收空间
- 支持表级别操作

数据文件结构

```
1 .ibd文件包含：
2   - 段 (Segment)：表或索引
3     - 区 (Extent)：1MB = 64个页
4       - 页 (Page)：16KB (默认)
5         - 行记录
```

关键配置参数

SQL

```
1  -- Buffer Pool大小
2  innodb_buffer_pool_size = 4G
3
4  -- Buffer Pool实例数 (减少锁竞争)
5  innodb_buffer_pool_instances = 8
6
7  -- Redo Log文件大小
8  innodb_log_file_size = 512M
9
10 -- Redo Log文件数量
11 innodb_log_files_in_group = 2
12
13 -- Flush策略
14 innodb_flush_log_at_trx_commit = 1  -- 最安全
15 -- 0：每秒刷盘，性能最好
16 -- 1：每次提交刷盘，最安全
17 -- 2：每次提交写OS缓存，每秒刷盘
18
19 -- IO容量限制
20 innodb_io_capacity = 200
21 innodb_io_capacity_max = 2000
22
23 -- Purge线程数
24 innodb_purge_threads = 4
```

CAUTION

`innodb_flush_log_at_trx_commit=1` 是最安全的设置，但性能较差。根据业务需求权衡。

13.3 MyISAM 存储引擎

MyISAM 是 MySQL 早期的默认引擎，现在已不推荐使用。

特性对比

特性	MyISAM	InnoDB
事务	X	✓

行级锁	X (只有表锁)	✓
外键	X	✓
MVCC	X	✓
崩溃恢复	X	✓
全文索引	✓ (5.6 前)	✓ (5.6+)
COUNT(*)	快 (有计数器)	慢 (需扫描)
存储空间	较小	较大

适用场景

适合：

- 只读或读多写少
- 不需要事务
- 简单的计数操作

不适合：

- 需要事务支持
- 高并发写入
- 数据安全性要求高

NOTE

MySQL 5.5 之后，InnoDB 成为默认引擎。新项目应该使用 InnoDB。

13.4 其他存储引擎

Memory 引擎

特点：

- 数据存储在内存中
- 速度极快
- 重启后数据丢失
- 支持哈希索引

```

1 CREATE TABLE sessions (
2     session_id VARCHAR(64) PRIMARY KEY,
3     data TEXT
4 ) ENGINE=Memory;

```

适用场景：临时表、缓存表

Archive 引擎

特点：

- 高压缩比
- 只支持 INSERT 和 SELECT
- 不支持 UPDATE 和 DELETE
- 适合归档数据

```
1 CREATE TABLE logs_archive (  
2     id INT AUTO_INCREMENT PRIMARY KEY,  
3     log_time DATETIME,  
4     message TEXT  
5 ) ENGINE=Archive;
```

CSV 引擎

特点:

- 数据以 CSV 格式存储
- 可以直接用文本编辑器查看
- 性能较差

适用场景: 数据交换

13.5 存储引擎的选择

场景	推荐引擎	原因
通用业务	InnoDB	功能全面, 支持事务
临时数据	Memory	速度快
日志归档	Archive	压缩率高
数据交换	CSV	易于导入导出
只读报表	MyISAM	简单计数快

TIP

95% 的场景都应该使用 InnoDB。只有在特殊需求下才考虑其他引擎。

14 PostgreSQL 特性和应用

PostgreSQL 是最先进的开源关系型数据库, 以功能丰富和标准兼容著称。

NOTE

PostgreSQL 的核心理念是扩展性和标准兼容性, 被称为“世界上最先进的开源关系型数据库”。

14.1 PostgreSQL vs MySQL

特性	PostgreSQL	MySQL
许可证	PostgreSQL License	GPL
SQL 标准	高度兼容	部分兼容
数据类型	丰富	一般
JSON 支持	优秀 (JSONB)	良好
GIS	PostGIS (最强)	一般

复制	流复制+逻辑复制	主从+组复制
分区表	声明式分区	支持
并行查询	支持	支持
社区活跃度	高	非常高
学习曲线	较陡	平缓

14.2 PostgreSQL 的核心特性

丰富的数据类型

SQL

```
1  -- 基本类型
2  CREATE TABLE example (
3      id SERIAL PRIMARY KEY,
4      name VARCHAR(100),
5      age INTEGER,
6      salary NUMERIC(10, 2),
7      is_active BOOLEAN,
8      created_at TIMESTAMP WITH TIME ZONE
9  );
10
11 -- 数组类型
12 CREATE TABLE products (
13     id SERIAL PRIMARY KEY,
14     name VARCHAR(100),
15     tags TEXT[], -- 文本数组
16     prices NUMERIC[] -- 数字数组
17 );
18
19 -- JSON/JSONB类型
20 CREATE TABLE documents (
21     id SERIAL PRIMARY KEY,
22     metadata JSONB -- 二进制JSON, 支持索引
23 );
24
25 -- 范围类型
26 CREATE TABLE events (
27     id SERIAL PRIMARY KEY,
28     duration TSRANGE -- 时间戳范围
29 );
30
31 -- 几何类型
32 CREATE TABLE locations (
33     id SERIAL PRIMARY KEY,
34     position POINT,
35     area POLYGON
36 );
37
38 -- UUID类型
39 CREATE TABLE users (
40     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
41     name VARCHAR(100)
42 );
```

高级索引

```
1  -- B-tree索引 (默认)
2  CREATE INDEX idx_name ON users(name);
3
4  -- GIN索引 (适合数组、JSONB、全文搜索)
5  CREATE INDEX idx_tags ON products USING GIN(tags);
6  CREATE INDEX idx_metadata ON documents USING GIN(metadata);
7
8  -- GiST索引 (适合几何数据、范围类型)
9  CREATE INDEX idx_position ON locations USING GIST(position);
10
11 -- BRIN索引 (适合大型有序表)
12 CREATE INDEX idx_created_at ON logs USING BRIN(created_at);
13
14 -- 部分索引
15 CREATE INDEX idx_active_users ON users(name) WHERE is_active = true;
16
17 -- 表达式索引
18 CREATE INDEX idx_lower_name ON users(LOWER(name));
```

TIP

PostgreSQL 的索引类型非常丰富，可以根据数据特点选择最合适的索引。

强大的 JSON 支持

```
1  -- 创建JSONB字段
2  CREATE TABLE products (
3      id SERIAL PRIMARY KEY,
4      name VARCHAR(100),
5      attributes JSONB
6  );
7
8  -- 插入JSON数据
9  INSERT INTO products (name, attributes) VALUES
10 ('Laptop', '{"brand": "Dell", "cpu": "i7", "ram": 16, "colors": ["black", "silver"]}');
11
12 -- 查询JSON字段
13 SELECT * FROM products
14 WHERE attributes->'brand' = 'Dell';
15
16 -- 嵌套查询
17 SELECT * FROM products
18 WHERE attributes->'cpu' ? 'i7';
19
20 -- 数组查询
21 SELECT * FROM products
22 WHERE attributes->'colors' ? 'black';
23
```

```
24 -- 创建GIN索引
25 CREATE INDEX idx_attributes ON products USING GIN(attributes);
```

窗口函数和 CTE

```
1  -- 窗口函数
2  SELECT
3      name,
4      department,
5      salary,
6      RANK() OVER (PARTITION BY department ORDER BY salary DESC) as rank,
7      AVG(salary) OVER (PARTITION BY department) as dept_avg
8  FROM employees;
9
10 -- 公用表表达式 (CTE)
11 WITH dept_stats AS (
12     SELECT
13         department,
14         COUNT(*) as emp_count,
15         AVG(salary) as avg_salary
16     FROM employees
17     GROUP BY department
18 )
19 SELECT * FROM dept_stats
20 WHERE emp_count > 10
21 ORDER BY avg_salary DESC;
22
23 -- 递归CTE
24 WITH RECURSIVE hierarchy AS (
25     -- 基础情况
26     SELECT id, name, manager_id, 1 as level
27     FROM employees
28     WHERE manager_id IS NULL
29
30     UNION ALL
31
32     -- 递归情况
33     SELECT e.id, e.name, e.manager_id, h.level + 1
34     FROM employees e
35     JOIN hierarchy h ON e.manager_id = h.id
36 )
37 SELECT * FROM hierarchy
38 ORDER BY level, name;
```

全文搜索

```
1  -- 创建表
2  CREATE TABLE articles (
3      id SERIAL PRIMARY KEY,
4      title VARCHAR(200),
5      content TEXT,
6      tsvector_content TSVECTOR
```

```
7 );
8
9 -- 生成tsvector
10 UPDATE articles
11 SET tsvector_content = to_tsvector('chinese', title || ' ' || content);
12
13 -- 创建GIN索引
14 CREATE INDEX idx_search ON articles USING GIN(tsvector_content);
15
16 -- 全文搜索
17 SELECT * FROM articles
18 WHERE tsvector_content @@ to_tsquery('chinese', '数据库 & 优化');
19
20 -- 排名
21 SELECT *, ts_rank(tsvector_content, query) as rank
22 FROM articles,
23      to_tsquery('chinese', '数据库') as query
24 WHERE tsvector_content @@ query
25 ORDER BY rank DESC;
```

NOTE

对于复杂的全文搜索需求，可以考虑使用 Elasticsearch，但 PostgreSQL 的内置功能已经能满足大部分场景。

分区表

```
1 -- 创建分区表
2 CREATE TABLE measurements (
3     id SERIAL,
4     city_id INT NOT NULL,
5     logdate DATE NOT NULL,
6     temperature INT
7 ) PARTITION BY RANGE (logdate);
8
9 -- 创建分区
10 CREATE TABLE measurements_2024_q1
11     PARTITION OF measurements
12     FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');
13
14 CREATE TABLE measurements_2024_q2
15     PARTITION OF measurements
16     FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');
17
18 -- 自动分区裁剪
19 EXPLAIN SELECT * FROM measurements
20 WHERE logdate >= '2024-01-01' AND logdate < '2024-04-01';
21 -- 只扫描 measurements_2024_q1 分区
```



扩展系统

PostgreSQL 支持丰富的扩展：

```

1  -- PostGIS (地理信息系统)
2  CREATE EXTENSION postgis;
3
4  -- pg_stat_statements (性能分析)
5  CREATE EXTENSION pg_stat_statements;
6
7  -- uuid-oss (UUID生成)
8  CREATE EXTENSION "uuid-oss";
9
10 -- hstore (键值对)
11 CREATE EXTENSION hstore;
12
13 -- 查看所有可用扩展
14 SELECT * FROM pg_available_extensions;

```

常用扩展：

扩展	用途
PostGIS	地理空间数据
pg_stat_statements	SQL 性能统计
pgcrypto	加密函数
citext	大小写不敏感文本
ltree	层次化数据
btree_gin	GIN 索引支持普通类型

14.3 PostgreSQL 的应用场景

GIS 应用

```

1  -- 使用PostGIS
2  SELECT name,
3         ST_Distance(
4             location,
5             ST_SetSRID(ST_MakePoint(116.4074, 39.9042), 4326)
6         ) as distance
7  FROM places
8  ORDER BY distance
9  LIMIT 10;

```

时序数据

```

1  -- 使用时序扩展
2  CREATE EXTENSION timescaledb;
3
4  -- 创建hypertable
5  CREATE TABLE sensor_data (
6      time TIMESTAMPTZ NOT NULL,
7      sensor_id INT,
8      value DOUBLE PRECISION

```

```
9 );
10
11 SELECT create_hypertable('sensor_data', 'time');
```

复杂数据分析

```
1 -- 复杂聚合
2 SELECT
3     department,
4     PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) as median_salary,
5     STDDEV(salary) as salary_stddev
6 FROM employees
7 GROUP BY department;
```

TIP

PostgreSQL 适合需要复杂查询、数据分析、GIS 等高级功能的场景。

15 关系型数据库性能调优

性能调优是一个系统工程，需要从多个维度综合考虑。

NOTE

性能调优的核心原则是先测量，后优化。没有监控就没有优化。

15.1 性能监控

MySQL 监控

```
1 -- 查看当前连接
2 SHOW PROCESSLIST;
3
4 -- 查看慢查询
5 SHOW VARIABLES LIKE 'slow_query_log';
6 SHOW VARIABLES LIKE 'long_query_time';
7
8 -- 查看InnoDB状态
9 SHOW ENGINE INNODB STATUS\G
10
11 -- 查看性能指标
12 SHOW STATUS LIKE 'Threads_connected';
13 SHOW STATUS LIKE 'Questions';
14 SHOW STATUS LIKE 'Innodb_buffer_pool_read%';
15
16 -- 使用performance_schema
17 SELECT * FROM performance_schema.events_statements_summary_by_digest
18 ORDER BY sum_timer_wait DESC
19 LIMIT 10;
```

PostgreSQL 监控

```
1  -- 查看当前活动
2  SELECT * FROM pg_stat_activity;
3
4  -- 查看慢查询
5  SELECT query, calls, total_time, mean_time
6  FROM pg_stat_statements
7  ORDER BY mean_time DESC
8  LIMIT 10;
9
10 -- 查看表统计
11 SELECT schemaname, relname, seq_scan, idx_scan
12 FROM pg_stat_user_tables
13 ORDER BY seq_scan DESC;
14
15 -- 查看索引使用
16 SELECT indexrelname, idx_scan, idx_tup_read
17 FROM pg_stat_user_indexes
18 ORDER BY idx_scan ASC;
```

15.2 SQL 优化

使用 EXPLAIN 分析

```
1  -- MySQL
2  EXPLAIN SELECT * FROM users WHERE age > 20;
3  EXPLAIN FORMAT=JSON SELECT * FROM users WHERE age > 20; -- 详细信息
4
5  -- PostgreSQL
6  EXPLAIN SELECT * FROM users WHERE age > 20;
7  EXPLAIN ANALYZE SELECT * FROM users WHERE age > 20; -- 实际执行
```

关注指标：

- `type`：访问类型（ALL < index < range < ref < eq_ref < const）
- `key`：使用的索引
- `rows`：扫描行数
- `Extra`：额外信息（Using filesort, Using temporary 等）

常见优化技巧

1. 避免 SELECT *

```
1  -- 不好
2  SELECT * FROM users;
3
4  -- 好
5  SELECT id, name, email FROM users;
```

2. 优化 JOIN

```
1 -- 确保JOIN字段有索引
2 -- 小表驱动大表
3 SELECT * FROM small_table s
4 JOIN large_table l ON s.id = l.small_id;
```

 SQL

3. 使用覆盖索引

```
1 -- 创建覆盖索引
2 CREATE INDEX idx_covering ON users(dept_id, name, email);
3
4 -- 查询只需访问索引
5 SELECT dept_id, name, email FROM users WHERE dept_id = 'CS01';
```

 SQL

4. 批量操作

```
1 -- 不好：逐条插入
2 INSERT INTO users VALUES (1, 'Alice');
3 INSERT INTO users VALUES (2, 'Bob');
4
5 -- 好：批量插入
6 INSERT INTO users VALUES
7     (1, 'Alice'),
8     (2, 'Bob'),
9     (3, 'Carol');
```

 SQL

5. 分页优化

```
1 -- 不好：深分页
2 SELECT * FROM users ORDER BY id LIMIT 10 OFFSET 10000;
3
4 -- 好：游标分页
5 SELECT * FROM users
6 WHERE id > last_seen_id
7 ORDER BY id
8 LIMIT 10;
```

 SQL

15.3 索引优化

识别缺失索引

```
1 -- MySQL：查看未使用索引的查询
2 SELECT * FROM sys.schema_statements_with_full_table_scans;
3
4 -- PostgreSQL：查看顺序扫描多的表
5 SELECT relname, seq_scan, seq_tup_read
6 FROM pg_stat_user_tables
7 WHERE seq_scan > 0
8 ORDER BY seq_tup_read DESC;
```

 SQL

识别冗余索引

```
1 -- MySQL
2 SELECT * FROM sys.schema_redundant_indexes;
3
4 -- PostgreSQL
5 SELECT * FROM pg_stat_user_indexes
6 WHERE idx_scan = 0;
```

索引维护

```
1 -- MySQL: 分析表
2 ANALYZE TABLE users;
3
4 -- PostgreSQL: 分析表
5 ANALYZE users;
6
7 -- 重建索引
8 ALTER INDEX idx_name REBUILD; -- PostgreSQL
9 OPTIMIZE TABLE users; -- MySQL
```

15.4 配置优化

MySQL 关键配置

```
1 # my.cnf
2
3 [mysqld]
4 # 连接
5 max_connections = 500
6 thread_cache_size = 64
7
8 # InnoDB
9 innodb_buffer_pool_size = 4G
10 innodb_buffer_pool_instances = 8
11 innodb_log_file_size = 512M
12 innodb_flush_log_at_trx_commit = 1
13 innodb_flush_method = O_DIRECT
14
15 # 查询
16 query_cache_type = 0 # MySQL 8.0已移除
17 sort_buffer_size = 2M
18 join_buffer_size = 2M
19
20 # 日志
21 slow_query_log = 1
22 long_query_time = 2
23 log_queries_not_using_indexes = 1
```

PostgreSQL 关键配置

```
1 # postgresql.conf
```

```
2
3 # 内存
4 shared_buffers = 2GB # 物理内存的25%
5 effective_cache_size = 6GB # 物理内存的75%
6 work_mem = 4MB
7 maintenance_work_mem = 512MB
8
9 # WAL
10 wal_buffers = 16MB
11 checkpoint_completion_target = 0.9
12
13 # 查询规划
14 random_page_cost = 1.1 # SSD设置为1.1
15 effective_io_concurrency = 200 # SSD
16
17 # 连接
18 max_connections = 200
```

CAUTION

配置调整需要根据实际硬件和业务负载进行测试，不要盲目套用。

15.5 架构优化

读写分离

```
1 应用程序
2  ↓
3  读写路由器
4  └─ 主库 (写)
5  └─ 从库1 (读)
6  └─ 从库2 (读)
7  └─ 从库N (读)
```

实现：

- MySQL：主从复制 + ProxySQL/MaxScale
- PostgreSQL：流复制 + Pgpool-II

分库分表

垂直拆分：

```
1 用户数据库: users, profiles, settings
2 订单数据库: orders, order_items, payments
3 商品数据库: products, categories, inventory
```

水平拆分：

```
1 -- 按用户ID取模
2 user_id % 4 = 0 → database_0
3 user_id % 4 = 1 → database_1
4 user_id % 4 = 2 → database_2
5 user_id % 4 = 3 → database_3
```

中间件：

- MyCat
- ShardingSphere
- Vitess

缓存层

1	请求流程：	text
2	应用 → Redis缓存 → 数据库	
3	↓ 命中	
4	返回结果	
5	↓ 未命中	
6	查询DB → 写入缓存	

缓存策略：

- Cache Aside（旁路缓存）
- Read Through
- Write Through
- Write Behind

TIP

缓存是提高性能最有效的手段之一，但要注意缓存一致性问题。

15.6 硬件优化

存储

- 使用 SSD/NVMe 替代 HDD
- RAID 10 提供性能和冗余
- 分离数据文件和日志文件到不同磁盘

内存

- 增加 RAM 提高缓存命中率
- 足够的内存可以避免 swap

CPU

- 多核提高并发处理能力
- 高主频提高单查询性能

网络

- 万兆网卡减少网络瓶颈
- 低延迟网络连接

15.7 性能调优流程

1. 建立基线

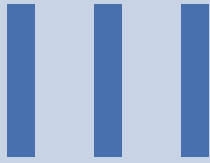
2	↓
3	2. 监控和测量
4	↓
5	3. 识别瓶颈
6	↓
7	4. 制定优化方案
8	↓
9	5. 实施优化
10	↓
11	6. 验证效果
12	↓
13	7. 回到步骤2（持续优化）

NOTE

性能调优是一个持续的过程，不是一次性的任务。



本章完



并发编程与分布式系统

4. 并发编程基础	117
5. 分布式系统理论	118

Chapter 4

并发编程基础

Chapter 5

分布式系统理论

分布式系统是由多台计算机通过网络协同工作，对外表现为一个统一系统的软件架构。它在现代互联网应用中无处不在，从搜索引擎到社交媒体，从电子商务到云计算平台。

NOTE

分布式系统的核心挑战在于不确定性：网络延迟、节点故障、时钟不同步等问题使得分布式系统比单机系统复杂得多。

1 分布式系统概述与挑战

1.1 什么是分布式系统

分布式系统是一组独立的计算机，对用户来说就像是一个单一的系统。

关键特征：

- 并发性：多个组件同时执行
- 缺乏全局时钟：没有统一的时序参考
- 独立故障：组件可能独立失败
- 消息传递：通过消息进行通信

1.2 为什么需要分布式系统

可扩展性 (Scalability)

- 垂直扩展：增加单机资源（CPU、内存）
- 水平扩展：增加机器数量
- 分布式系统支持水平扩展，理论上可以无限扩展

可用性 (Availability)

- 单点故障会导致整个系统不可用
- 分布式系统通过冗余提高可用性
- 即使部分节点故障，系统仍可运行

性能 (Performance)

- 地理分布减少延迟
- 并行处理提高吞吐量
- 负载均衡优化资源利用

容错性 (Fault Tolerance)

- 冗余设计避免单点故障
- 自动故障检测和恢复
- 数据复制保证可靠性

1.3 分布式系统的挑战

网络问题

- 网络延迟：消息传输需要时间
- 网络分区：网络可能分裂成多个部分
- 带宽限制：数据传输速率有限
- 不可靠性：消息可能丢失、重复或乱序

CAUTION

网络是不可靠的！永远不要假设消息一定会到达，也不要假设只会到达一次。

时钟同步

- 不同节点的时钟可能不同步
- 无法确定事件的绝对顺序
- 需要使用逻辑时钟或向量时钟

一致性问题

- 如何在多个副本之间保持一致
- CAP 定理的限制
- 一致性 vs 可用性的权衡

部分故障

- 某些节点可能故障，其他节点正常
- 难以区分是节点故障还是网络延迟
- 需要复杂的故障检测机制

并发控制

- 多个客户端同时访问共享资源
- 需要分布式锁或事务
- 死锁检测和处理更复杂

TIP

Leslie Lamport 的名言：“分布式系统是这样一个系统，其中你不知道的计算机发生故障，导致你无法完成自己的工作。”

2 CAP 定理详解

CAP 定理是分布式系统理论的基石，由 Eric Brewer 于 2000 年提出，并在 2002 年由 Seth Gilbert 和 Nancy Lynch 证明。

2.1 CAP 三要素

一致性 (Consistency)

定义：所有节点在同一时刻看到的数据是相同的。

特点：

- 读操作总是返回最新的写操作结果
- 强一致性保证
- 类似于单机数据库的 ACID 中的 C

示例：

- 1 客户端A写入 $X = 1$
- 2 客户端B立即读取 X ，必须得到 1

可用性 (Availability)

定义：每个请求都能在合理时间内收到响应（成功或失败）。

特点：

- 系统始终可用，不会拒绝服务
- 不保证返回最新数据
- 关注响应时间

示例：

- 1 客户端发送请求
- 2 系统必须在超时时间内返回响应
- 3 即使返回的是旧数据，也算可用

分区容错性 (Partition Tolerance)

定义：系统在遇到网络分区时仍能继续运行。

特点：

- 网络分区是不可避免的
- 系统必须能够容忍分区
- 实际分布式系统中 P 是必须的

示例：

- 1 网络分成两部分：{A, B} 和 {C, D}
- 2 两部分之间无法通信
- 3 系统仍需在各分区内正常工作

NOTE

在现实世界中，网络分区是必然发生的（路由器故障、光缆被挖断等），因此 P 是分布式系统必须具备的特性。

2.2 CAP 定理的核心结论

一个分布式系统最多只能同时满足 CAP 中的两个特性。

1	C (一致性)	text
2	/ \	
3	/ \	
4	/ \	
5	A-----P	
6	(可用性) (分区容错性)	
7		
8	只能选择两条边，不能三者兼得	

2.3 三种组合

CP 系统（一致性 + 分区容错性）

特点：

- 保证数据一致性
- 发生分区时，不可用的分区会拒绝服务
- 牺牲可用性来保证一致性

典型应用：

- 银行系统
- 分布式数据库（HBase、MongoDB 默认配置）
- ZooKeeper
- etcd

场景示例：

1	网络分区发生
2	主节点在分区A，从节点在分区B
3	分区B的节点拒绝读写请求（保证一致性）
4	只有分区A可以正常服务

AP 系统（可用性 + 分区容错性）

特点：

- 保证系统始终可用
- 允许数据暂时不一致
- 最终会达到一致状态

典型应用：

- DNS 系统
- Cassandra
- DynamoDB
- CDN

场景示例：

1	网络分区发生
---	--------

- 2 两个分区都可以接受写请求
- 3 可能导致数据冲突
- 4 分区恢复后通过某种策略解决冲突

CA 系统（一致性 + 可用性）

特点：

- 理论上存在，但实际中不存在
- 因为没有分区容错性
- 只能在单机或非分布式系统中实现

CAUTION

CA 系统在实际的分布式系统中是不存在的，因为网络分区是不可避免的。任何声称是 CA 的“分布式系统”实际上都不是真正的分布式系统。

2.4 CAP 的实际意义

CAP 定理告诉我们：

1. 没有完美的系统：必须做出权衡
2. 根据业务需求选择：不同场景需要不同的权衡
3. 动态调整：可以在不同情况下选择不同的策略
4. 理解代价：知道选择某个特性意味着放弃什么

TIP

大多数现代分布式系统都是 AP 或 CP，具体选择取决于业务需求。例如，社交网络倾向于 AP（可以接受短暂的不一致），而金融系统倾向于 CP（必须保证一致性）。

3 CAP 权衡与实际应用

3.1 如何选择合适的 CAP 组合

考虑因素

业务需求

- 金融交易：需要 CP（一致性优先）
- 社交动态：可以 AP（可用性优先）
- 电商库存：需要 CP（避免超卖）
- 用户资料：可以 AP（最终一致即可）

用户体验

- 实时性要求高：可能需要 CP
- 可以接受延迟：可以选择 AP
- 离线可用：需要 AP

技术约束

- 数据量大小
- 读写比例
- 地理分布
- 团队技术能力

3.2 实际应用案例

案例 1：电商平台

需求分析：

- 商品浏览：AP（高可用，可以接受短暂不一致）
- 购物车：AP（用户可以稍后同步）
- 订单创建：CP（必须保证一致性，避免重复订单）
- 库存扣减：CP（避免超卖）
- 支付处理：CP（绝对不能出错）

解决方案：

- 1 使用混合架构：
- 2 - 商品缓存：Redis（AP）
- 3 - 订单系统：关系型数据库 + 分布式事务（CP）
- 4 - 库存系统：Redis + 数据库最终一致（AP -> CP）
- 5 - 支付系统：强一致性数据库（CP）

案例 2：社交网络

需求分析：

- 发布动态：AP（快速响应，最终一致）
- 点赞评论：AP（可以接受短暂不一致）
- 好友关系：CP（需要一致性）
- 消息推送：AP（允许丢失少量消息）

解决方案：

- 1 - 动态存储：Cassandra（AP）
- 2 - 关系数据：MySQL主从（最终一致）
- 3 - 实时消息：Kafka + WebSocket（AP）

案例 3：协作编辑工具

需求分析：

- 文档编辑：需要高可用和低延迟
- 多人协作：需要解决冲突
- 历史版本：需要一致性

解决方案：

- 1 - 使用CRDT（无冲突复制数据类型）
- 2 - 操作转换（OT）算法
- 3 - 最终一致性模型
- 4 - 定期快照保证长期一致性

NOTE

Google Docs、Figma 等协作工具都使用了类似的技术来实现多人实时协作。

3.3 超越 CAP: PACELC 定理

CAP 定理只考虑了网络分区的情况，PACELC 定理进行了扩展：

If Partition (P), then trade-off between Availability (A) and Consistency (C); Else (E), trade-off between Latency (L) and Consistency (C).

含义：

- 如果发生分区 (P)，在可用性 (A) 和一致性 (C) 之间权衡
- 否则 (E)，在延迟 (L) 和一致性 (C) 之间权衡

意义：

- 即使没有分区，也需要在延迟和一致性之间权衡
- 更全面地描述了分布式系统的权衡

TIP

PACELC 定理提醒我们，即使在正常情况下，也需要考虑延迟和一致性的平衡，而不仅仅是分区时的选择。

4 BASE 理论

BASE 理论是对 CAP 中 AP 系统的进一步阐述，由 eBay 的 Dan Pritchett 提出。

4.1 BASE 三要素

基本可用 (Basically Available)

定义：分布式系统在出现故障时，允许损失部分可用性，但保证核心功能可用。

特点：

- 不是 100% 可用，而是“基本”可用
- 允许响应时间增加
- 允许功能降级

示例：

- 1 双11购物节：
- 2 - 核心功能（下单、支付）保持可用
- 3 - 非核心功能（推荐、评价）可能降级
- 4 - 响应时间可能变慢，但不会完全不可用

软状态 (Soft State)

定义：允许系统中的数据存在中间状态，这些状态不影响系统整体可用性。

特点：

- 数据可以有临时不一致
- 状态会随着时间推移而变化

- 不需要立即达到一致

示例：

- 1 用户修改头像：
- 2 1. 上传到新服务器A
- 3 2. CDN节点B仍然是旧头像
- 4 3. 几分钟后，CDN同步完成
- 5 4. 期间存在“软状态”

最终一致性（Eventual Consistency）

定义：系统中的数据副本在经过一段时间后，最终会达到一致状态。

特点：

- 不保证即时一致
- 保证最终会一致
- 一致性窗口期内的不一致是可接受的

示例：

- 1 DNS更新：
- 2 - 修改DNS记录
- 3 - 全球DNS服务器需要时间传播
- 4 - 可能在几分钟到几小时内不一致
- 5 - 但最终所有服务器都会更新

NOTE

BASE 理论的核心思想是：通过牺牲强一致性来获得高可用性和高性能，适用于大多数互联网应用场景。

4.2 最终一致性的实现方式

读写一致性级别

读己之所写（Read-your-writes）

用户总能读取到自己写入的最新数据。

实现：

- 将用户的读写请求路由到同一节点
- 或使用 session 粘滞

会话一致性（Session Consistency）

在同一会话中，保证读己之所写。

实现：

- 基于 session 的路由
- 客户端缓存

单调读一致性（Monotonic Read Consistency）

如果用户已经读取过某个数据的值，后续的读取不会返回更早的值。

实现:

- 版本向量
- 因果依赖跟踪

单调写一致性 (Monotonic Write Consistency)

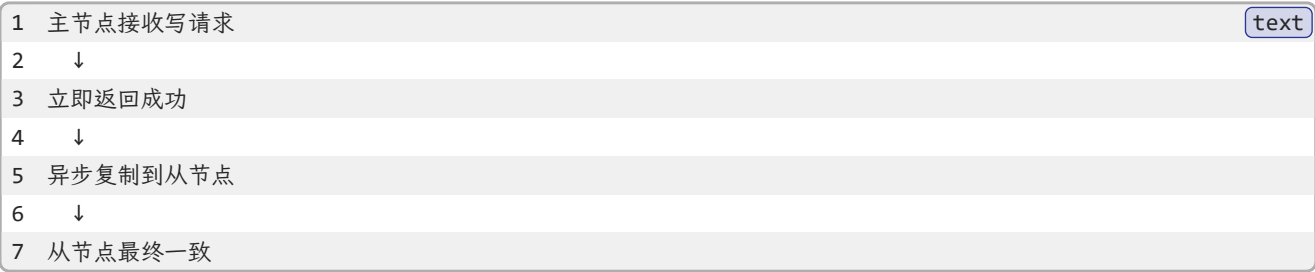
保证写操作的顺序执行。

实现:

- 队列化写操作
- 序列号

常见实现模式

异步复制



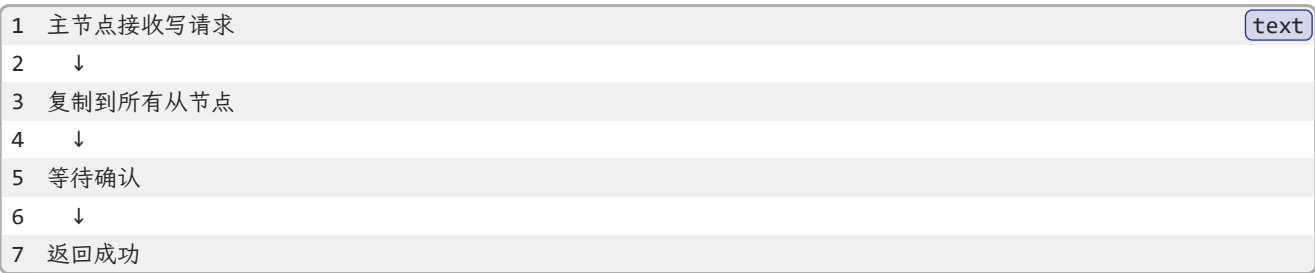
优点:

- 写性能好
- 延迟低

缺点:

- 可能丢失数据 (主节点故障)
- 读取可能不一致

同步复制



优点:

- 数据安全性高
- 一致性强

缺点:

- 写性能差
- 延迟高

半同步复制

折中方案：

- 至少复制到一个从节点
- 然后返回成功
- 平衡性能和安全性

TIP

MySQL 的主从复制默认是异步的，可以配置为半同步以提高数据安全性。

5 一致性模型

一致性模型定义了分布式系统中数据访问的行为规范。

5.1 强一致性模型

1 线性一致性 (Linearizability)

定义：所有操作看起来是原子性地在一个时间点执行，且符合实时顺序。

特点：

- 最强的一致性保证
- 等价于单线程执行
- 实现成本高

示例：

```
1 时间线:  $t_1 < t_2 < t_3$ 
2
3 t1: 客户端A写入  $X = 1$ 
4 t2: 客户端B读取  $X$ , 必须得到 1
5 t3: 客户端C读取  $X$ , 必须得到 1
```

应用：

- 分布式锁
- 领导者选举
- ZooKeeper、etcd

1 顺序一致性 (Sequential Consistency)

定义：所有进程看到的操作顺序相同，且符合程序顺序。

特点：

- 比线性一致性弱
- 不要求实时性
- 更容易实现

示例：

```
1 进程P1:  $W(x)1, W(y)1$ 
2 进程P2:  $R(y)1, R(x)1$ 
3
4 所有进程看到的顺序必须是:  $W(x)1, W(y)1, R(y)1, R(x)1$ 
```

5 但不能保证这个顺序与实际时间对应

5.2 弱一致性模型

因果一致性 (Causal Consistency)

定义：有因果关系的操作必须按顺序执行，无关操作可以任意顺序。

特点：

- 比顺序一致性弱
- 保留因果关系
- 性能较好

示例：

1	因果关系：
2	A发帖 -> B评论 -> C回复
3	
4	这三个操作必须按顺序可见
5	
6	但A发帖和D发帖（无关）可以任意顺序

实现：

- 向量时钟
- 依赖跟踪

最终一致性 (Eventual Consistency)

定义：如果没有新的更新，最终所有副本会达到一致状态。

特点：

- 最弱的一致性保证
- 最好的性能和可用性
- 可能存在冲突

应用：

- DNS
- CDN
- NoSQL 数据库（Cassandra、DynamoDB）

5.3 一致性模型对比

模型	强度	性能	实现难度	典型应用
线性一致性	最强	最低	最难	分布式锁
顺序一致性	强	低	难	共享内存
因果一致性	中	中	中	协作编辑
最终一致性	弱	最高	最简单	DNS、CDN

TIP
选择一致性模型时，需要根据业务需求权衡。不是越强越好，而是要找到最适合的平衡点。

6 分布式事务

分布式事务确保跨多个节点的操作要么全部成功，要么全部失败，保持数据一致性。

6.1 ACID vs BASE

特性	ACID	BASE
一致性	强一致性	最终一致性
可用性	较低	较高
性能	较低	较高
适用场景	金融、传统企业	互联网、大数据
典型系统	关系型数据库	NoSQL、分布式系统

6.2 两阶段提交（2PC）

2PC 是最经典的分布式事务协议。

I 工作原理

参与者角色：

- 协调者（Coordinator）：负责协调事务
- 参与者（Participant）：执行具体操作

两个阶段：

第一阶段：准备阶段（Prepare Phase）

1	1. 协调者向所有参与者发送PREPARE请求
2	2. 参与者执行事务操作，但不提交
3	3. 参与者写入undo log和redo log
4	4. 参与者返回YES或NO
5	- YES：准备就绪，可以提交
6	- NO：准备失败，需要回滚

第二阶段：提交阶段（Commit Phase）

1	情况1：所有参与者都返回YES
2	1. 协调者发送COMMIT请求
3	2. 参与者正式提交事务
4	3. 释放资源
5	4. 返回ACK
6	
7	情况2：有参与者返回NO或超时
8	1. 协调者发送ROLLBACK请求
9	2. 参与者回滚事务
10	3. 释放资源
11	4. 返回ACK

1	协调者	参与者A	参与者B	text
2				
3	--- PREPARE ----->			
4	--- PREPARE ----->			
5				
6	<-- YES -----			
7	<-- YES -----			
8				
9	--- COMMIT ----->			
10	--- COMMIT ----->			
11				
12	<-- ACK -----			
13	<-- ACK -----			

优点

- 保证了强一致性
- 原理简单，易于理解
- 广泛应用

缺点

- 同步阻塞：所有参与者在等待期间都持有资源
- 单点故障：协调者故障会导致整个事务不确定
- 数据不一致：网络异常可能导致部分提交
- 性能问题：两次网络往返，延迟高

CAUTION

2PC 在生产环境中很少直接使用，因为它的性能问题和可靠性问题。通常作为更高级协议的基础。

6.3 三阶段提交（3PC）

3PC 在 2PC 的基础上增加了超时机制和一个预提交阶段。

三个阶段

CanCommit 阶段

协调者询问参与者是否可以执行事务。

PreCommit 阶段

参与者执行事务但不提交，进入准备状态。

DoCommit 阶段

正式提交或回滚。

改进点

- 增加了超时机制

- 减少了阻塞时间
- 降低了单点故障的影响

仍然存在的问题

- 网络分区时仍可能不一致
- 复杂度更高
- 实际应用较少

NOTE

3PC 理论上比 2PC 更好，但由于复杂度和实际问题，在实际系统中使用不多。

6.4 TCC (Try-Confirm-Cancel)

TCC 是一种应用层的补偿事务协议。

三个操作

Try (尝试)

- 检查业务活动是否可以执行
- 预留业务资源
- 不真正执行业务

Confirm (确认)

- 真正执行业务
- 不使用任何业务检查
- 幂等操作

Cancel (取消)

- 释放 Try 阶段预留的资源
- 补偿操作
- 幂等操作

示例：转账

```
1  # Try阶段
2  def try_transfer(from_account, to_account, amount):
3      # 检查余额
4      if from_account.balance < amount:
5          return False
6      # 冻结资金
7      from_account.frozen_balance += amount
8      return True
9
10 # Confirm阶段
11 def confirm_transfer(from_account, to_account, amount):
12     # 真正转账
13     from_account.balance -= amount
```

 Python

```
14     from_account.frozen_balance -= amount
15     to_account.balance += amount
16
17 # Cancel阶段
18 def cancel_transfer(from_account, amount):
19     # 解冻资金
20     from_account.frozen_balance -= amount
```

优点:

- 应用层控制, 灵活
- 避免了长时间锁定资源
- 适合业务流程复杂的场景

缺点:

- 需要实现三个接口, 开发成本高
- 需要考虑幂等性
- 空回滚、悬挂等问题需要处理

TIP

TCC 适合对一致性要求高、业务流程复杂的场景, 如金融系统、电商订单等。

6.5 Saga 模式

Saga 将长事务拆分为多个本地短事务, 每个事务都有对应的补偿操作。

I 两种编排方式

choreography (编舞式)

每个事务完成后发布事件, 触发下一个事务。

```
1  订单服务 --创建订单--> 事件总线
2
3          ↓
3  库存服务 <--订阅事件--
4          --扣减库存--> 事件总线
5
6          ↓
6  支付服务 <--订阅事件--
7          --发起支付-->
```

优点:

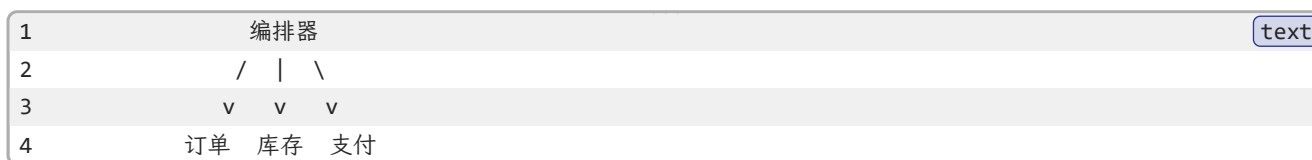
- 去中心化
- 松耦合

缺点:

- 流程难以追踪
- 循环依赖风险

Orchestration (编排式)

由一个编排器 (Orchestrator) 控制整个流程。



优点:

- 集中控制
- 易于监控和管理

缺点:

- 编排器可能成为瓶颈
- 耦合度较高

示例：电商订单

```
1 class OrderSaga:
2     def execute(self, order):
3         try:
4             # 步骤1: 创建订单
5             self.create_order(order)
6
7             # 步骤2: 扣减库存
8             self.deduct_inventory(order.items)
9
10            # 步骤3: 发起支付
11            self.initiate_payment(order)
12
13            # 步骤4: 更新订单状态
14            self.update_order_status(order, 'PAID')
15
16        except Exception as e:
17            # 补偿操作 (反向执行)
18            self.compensate(order)
19
20        def compensate(self, order):
21            # 退款
22            self.refund_payment(order)
23
24            # 恢复库存
25            self.restore_inventory(order.items)
26
27            # 取消订单
28            self.cancel_order(order)
```

优点:

- 适合长事务
- 每个步骤都是本地事务
- 灵活性高

缺点:

- 可能出现中间状态
- 补偿逻辑复杂

- 不保证隔离性

NOTE

Saga 模式在微服务架构中非常流行，特别适合业务流程长的场景。

6.6 分布式事务选型建议

协议	一致性	性能	复杂度	适用场景
2PC	强	低	中	传统企业系统
3PC	强	中	高	较少使用
TCC	最终	高	高	金融、电商
Saga	最终	高	中	微服务、长事务
本地消息表	最终	高	低	异步解耦
最大努力通知	最终	高	低	通知类业务

TIP

现代分布式系统更多使用最终一致性模型（TCC、Saga），而非强一致性模型（2PC、3PC）。

7 分布式锁实现

分布式锁用于在分布式环境中协调对共享资源的访问。

7.1 分布式锁的要求

- 互斥性：同一时刻只有一个客户端能持有锁
- 安全性：只有加锁的客户端才能解锁
- 死锁避免：即使客户端崩溃，锁也能释放
- 容错性：部分节点故障不影响锁的可用性

7.2 基于 Redis 的分布式锁

Redlock 算法

Redis 作者 antirez 提出的分布式锁算法。

实现步骤：

```
1 import redis
2 import time
3 import uuid
4
5 class Redlock:
6     def __init__(self, redis_nodes):
7         self.nodes = [redis.Redis(node) for node in redis_nodes]
8         self.quorum = len(redis_nodes) // 2 + 1
9         self.validity_time = 10000 # 锁有效期（毫秒）
10
```

Python

```
11 def acquire_lock(self, resource, ttl):
12     token = str(uuid.uuid4())
13     start_time = int(time.time() * 1000)
14
15     # 尝试在大多数节点上加锁
16     locked_nodes = 0
17     for node in self.nodes:
18         if node.set(resource, token, nx=True, px=ttl):
19             locked_nodes += 1
20
21     # 计算有效时间
22     elapsed = int(time.time() * 1000) - start_time
23     validity = ttl - elapsed
24
25     # 判断是否成功
26     if locked_nodes >= self.quorum and validity > 0:
27         return {'token': token, 'validity': validity}
28     else:
29         # 失败则释放所有已加的锁
30         self.release_lock(resource, token)
31         return None
32
33 def release_lock(self, resource, token):
34     # 使用Lua脚本保证原子性
35     lua_script = """
36     if redis.call("get", KEYS[1]) == ARGV[1] then
37         return redis.call("del", KEYS[1])
38     else
39         return 0
40     end
41     """
42     for node in self.nodes:
43         node.eval(lua_script, 1, resource, token)
```

优点:

- 性能高
- 实现相对简单

缺点:

- 依赖时钟同步
- 网络延迟可能导致问题
- 需要多个 Redis 实例

CAUTION

Martin Kleppmann 曾对 Redlock 提出质疑, 认为在极端情况下可能不安全。实际使用时需要仔细评估风险。

Redisson 实现

Java 中最常用的 Redis 分布式锁实现:

```
1 RLock lock = redissonClient.getLock("myLock");
```

 Java

```
2  try {
3      // 尝试加锁，最多等待10秒，锁定30秒后自动释放
4      boolean res = lock.tryLock(10, 30, TimeUnit.SECONDS);
5      if (res) {
6          // 执行业务逻辑
7      }
8  } finally {
9      lock.unlock();
10 }
```

特性：

- 看门狗机制（自动续期）
- 可重入
- 公平锁
- 读写锁

7.3 基于 ZooKeeper 的分布式锁

实现原理

利用 ZooKeeper 的临时顺序节点特性。

步骤：

1. 创建临时顺序节点 /locks/lock-0000000001
2. 获取 /locks 下所有子节点
3. 判断自己是否是最小的节点
 - 是：获得锁
 - 否：监听前一个节点
4. 前一个节点删除时，重新检查
5. 释放锁：删除自己的节点

```
1  from kazoo.client import KazooClient
2
3  class ZkDistributedLock:
4      def __init__(self, zk_hosts, lock_path):
5          self.zk = KazooClient(hosts=zk_hosts)
6          self.zk.start()
7          self.lock_path = lock_path
8          self.lock_node = None
9
10     def acquire(self, timeout=10):
11         # 创建临时顺序节点
12         self.lock_node = self.zk.create(
13             self.lock_path + "/lock-",
14             ephemeral=True,
15             sequence=True
16         )
17
18         # 获取所有子节点并排序
19         children = sorted(self.zk.get_children(self.lock_path))
20         my_node = self.lock_node.split('/')[-1]
21         my_index = children.index(my_node)
```

 Python

```
22
23     if my_index == 0:
24         # 我是第一个，获得锁
25         return True
26     else:
27         # 监听前一个节点
28         prev_node = children[my_index - 1]
29         event = self.zk.DataWatch(
30             self.lock_path + "/" + prev_node
31         )
32         # 等待前一个节点删除
33         event.wait(timeout=timeout)
34         return True
35
36     def release(self):
37         if self.lock_node:
38             self.zk.delete(self.lock_node)
39             self.lock_node = None
```

优点：

- 可靠性高
- 不会出现脑裂
- ZooKeeper 本身提供一致性保证

缺点：

- 性能相对较低
- 频繁创建删除节点
- 羊群效应（大量客户端同时监听）

TIP

Curator 框架提供了完善的 ZooKeeper 分布式锁实现，推荐使用。

7.4 基于 etcd 的分布式锁

etcd 提供了原生的租约（Lease）和锁机制。

```
1  import etcd3
2
3  client = etcd3.client()
4
5  # 创建锁
6  lock = client.lock('my-lock')
7
8  # 获取锁
9  if lock.acquire(timeout=10):
10     try:
11         # 执行业务逻辑
12         pass
13     finally:
14         # 释放锁
15         lock.release()
```

 Python

优点:

- API 简洁
- 基于 Raft，一致性强
- 适合云原生环境

7.5 分布式锁对比

实现方式	性能	可靠性	复杂度	适用场景
Redis Redlock	高	中	中	高性能要求
ZooKeeper	中	高	中	强一致性要求
etcd	中	高	低	K8s、云原生
数据库	低	高	低	简单场景

NOTE

选择分布式锁实现时，需要权衡性能、可靠性和复杂度。对于关键业务，建议使用 ZooKeeper 或 etcd；对于高性能场景，可以考虑 Redis。

8 Leader 选举算法

Leader 选举是分布式系统中的常见问题，用于选出一个协调者。

8.1 为什么需要 Leader 选举

- 协调任务：需要一个节点来协调分布式操作
- 避免冲突：防止多个节点同时执行相同操作
- 简化设计：主从架构比多主架构更简单
- 数据一致性：通过单一写入点保证一致性

8.2 Bully 算法

最简单的 Leader 选举算法。

算法流程

- 1
1. 当发现Leader失效时，发起选举
- 2
2. 向所有ID比自己大的节点发送ELECTION消息
- 3
3. 如果没有响应，自己成为Leader
- 4
4. 如果有响应，等待对方的COORDINATOR消息
- 5
5. 收到COORDINATOR消息，承认对方为Leader

优点:

- 简单易懂
- 保证选出 ID 最大的节点

缺点:

- 消息复杂度高 $O(n^2)$

- 网络不稳定时可能频繁选举
- ID 需要全局唯一且有序

8.3 Ring 算法

节点组织成环状结构。

算法流程

1. 检测到Leader失效
2. 向环中下一个节点发送ELECTION消息
3. 消息中包含自己的ID
4. 每个节点比较ID，保留最大的
5. 消息回到发起者时，最大ID成为Leader

优点：

- 消息复杂度 $O(n)$
- 比 Bully 算法高效

缺点：

- 环维护复杂
- 单点故障影响大

8.4 Raft 选举

Raft 算法中的 Leader 选举机制。

选举过程

1. 节点启动时为Follower
2. 如果在选举超时时间内没收到Leader心跳
3. 转为Candidate，增加term
4. 向所有节点请求投票
5. 如果获得多数票，成为Leader
6. 开始发送心跳

特点：

- 基于多数派
- 随机超时避免选票分裂
- term 机制防止脑裂

TIP

Raft 是目前最流行的共识算法之一，etcd、Consul 等都使用 Raft。

8.5 ZooKeeper 的 Leader 选举

基于 ZAB 协议的选举机制。

I 选举过程

1. 每个服务器提出自己的ZXID和SID
2. 比较ZXID, 较大的优先
3. ZXID相同, 比较SID, 较大的优先
4. 统计投票, 超过半数成为Leader
5. 进入同步阶段

特点:

- 基于 ZXID (事务 ID)
- 保证数据最新的节点成为 Leader
- 快速故障转移

9 拜占庭将军问题

拜占庭将军问题是分布式系统中的一个经典问题, 由 Leslie Lamport 等人于 1982 年提出。

9.1 问题描述

场景:

- N 个将军围攻一座城市
- 将军们需要通过信使传递消息来协调行动
- 有些将军可能是叛徒, 会发送错误消息
- 忠诚的将军需要达成一致的行动方案

核心问题:

在存在恶意节点的情况下, 如何让诚实节点达成一致?

9.2 形式化定义

拜占庭故障:

- 节点可能以任意方式故障
- 可能发送错误消息
- 可能故意破坏系统
- 比崩溃故障更难处理

要求:

1. 所有诚实节点达成一致的决策
2. 如果所有诚实节点初始值相同, 则决策为该值

9.3 解决方案

I PBFT (实用拜占庭容错)

Miguel Castro 和 Barbara Liskov 于 1999 年提出。

前提条件:

- 总节点数 $n \geq 3f + 1$ (f 为故障节点数)
- 异步网络环境

- 节点身份已知

三个阶段：

Pre-prepare 阶段

主节点接收请求，广播给所有备份节点。

Prepare 阶段

备份节点验证消息，广播 prepare 消息。

Commit 阶段

节点收集到足够多的 prepare 消息后，广播 commit 消息。

1	客户端	主节点	备份节点	text
2				
3		--REQUEST--->		
4			--PRE-PREPARE->	
5			<---PREPARE----	
6			--PREPARE----->	
7			<---COMMIT-----	
8			--COMMIT----->	
9		<---REPLY-----		

优点：

- 解决了拜占庭问题
- 性能相对较好

缺点：

- 消息复杂度高 $O(n^2)$
- 只能容忍 $< n/3$ 的故障节点
- 扩展性差

NOTE

PBFT 主要用于联盟链等小规模区块链系统，不适合大规模公有链。

PoW（工作量证明）

比特币使用的共识机制。

原理：

- 通过计算哈希 puzzle 来达成共识
- 需要消耗大量计算资源
- 最长链原则

优点：

- 完全去中心化
- 安全性高
- 无需许可

缺点:

- 能耗巨大
- 吞吐量低
- 确认时间长

■ PoS (权益证明)

以太坊 2.0 采用的机制。

原理:

- 根据持币数量和时长选择验证者
- 验证者质押代币
- 作恶会被罚没

优点:

- 能耗低
- 吞吐量较高
- 更环保

缺点:

- 可能存在富者愈富
- Nothing at Stake 问题

9.4 拜占庭容错的应用

■ 区块链

- 比特币: PoW
- 以太坊: PoS (2.0)
- Hyperledger Fabric: PBFT 变种

■ 航空航天

- 飞机控制系统
- 卫星通信
- 需要极高可靠性

■ 金融系统

- 跨境支付
- 证券交易
- 需要防篡改

TIP

拜占庭容错在开放、不可信的环境中特别重要，如公有区块链。而在可信环境中（如企业内部），通常不需要考虑拜占庭故障。

10 FL 不可能定理

FLP 不可能定理由 Fischer、Lynch 和 Paterson 于 1985 年证明。

10.1 定理内容

在异步分布式系统中，即使只有一个进程可能崩溃，也不存在一个确定性算法能够在有限时间内达成一致。

关键假设：

- 异步系统：消息延迟无上限
- 可能发生崩溃故障
- 确定性算法
- 需要在有限时间内终止

10.2 直观理解

为什么不可能：

1	1. 在异步系统中，无法区分：
2	- 节点很慢
3	- 节点已崩溃
4	
5	2. 如果等待太久，可能永远无法决定
6	3. 如果不等待，可能错过关键信息
7	4. 因此无法保证在有限时间内达成一致

10.3 实际意义

理论意义

- 证明了分布式共识的根本困难
- 解释了为什么需要妥协

实践指导

如何应对：

1. 引入随机化：
 - 随机化算法可以绕过 FLP
 - 如 Ben-Or 算法
2. 使用故障检测器：
 - 不完美的故障检测器
 - 最终准确的检测器
3. 接受概率性：
 - 不保证一定成功
 - 但以高概率成功
4. 部分同步模型：
 - 假设最终会同步
 - 如 Raft、Paxos

NOTE

FLP 定理并不意味着分布式共识不可能，而是说在纯异步系统中无法保证确定性地在规定时间内完成。实际系统通过各种方式绕过这个限制。

10.4 与 CAP 的关系

FLP 和 CAP 都揭示了分布式系统的根本限制：

- FLP：异步系统中无法保证一致性 + 可用性 + 终止性
- CAP：分区发生时无法保证一致性 + 可用性

两者都说明：**分布式系统必须做出权衡。**

TIP

理解 FLP 定理有助于我们认识到分布式系统的局限性，从而设计更合理的系统和协议。

11 总结

分布式系统理论为我们提供了理解和设计分布式系统的理论基础：

- CAP 定理：帮助我们理解一致性和可用性的权衡
- BASE 理论：提供了实现高可用性的思路
- 一致性模型：定义了不同强度的一致性保证
- 分布式事务：提供了跨节点保持一致性的方法
- 分布式锁：实现了分布式环境下的互斥访问
- Leader 选举：解决了协调者选择问题
- 拜占庭容错：处理恶意节点的场景
- FLP 定理：揭示了分布式共识的根本限制

本章完

IV

设计模式



计算机基础综合

Chapter 6

正则表达式

正则表达式 (Regular Expression, 简称 Regex) 是一种强大的文本模式匹配工具, 用于搜索、替换、验证和提取字符串中的特定模式。它被广泛应用于编程语言、文本编辑器、数据库和命令行工具中。

NOTE

正则表达式的核心理念是用简洁的模式描述复杂的文本规则。虽然学习曲线较陡, 但一旦掌握, 将成为处理文本的利器。

1 基础概念

1.1 什么是正则表达式

正则表达式是由普通字符和特殊字符 (元字符) 组成的模式字符串。

应用场景:

- 数据验证 (邮箱、手机号、身份证号)
- 文本搜索和替换
- 信息提取 (从日志中提取 IP 地址)
- 数据清洗 (去除 HTML 标签)
- 语法高亮

1.2 正则引擎

不同的语言和工具使用不同的正则引擎:

引擎类型	特点	代表实现
DFA	速度快, 不支持回溯	grep, awk
NFA	功能强大, 支持回溯	Perl, Python, Java
POSIX NFA	标准兼容性好	Unix 工具
PCRE	功能最丰富	PHP, Apache

TIP

大多数现代编程语言使用 NFA 引擎, 支持更复杂的特性, 但需要注意性能问题。

2 基本语法

2.1 普通字符

大多数字母和数字都是普通字符，直接匹配自身：

1	abc	匹配 "abc"	text
2	123	匹配 "123"	
3	Hello	匹配 "Hello"	

2.2 元字符

元字符在正则中有特殊含义，需要转义才能匹配字面值。

单个字符匹配

元字符	含义	示例	匹配
<code>\.</code>	任意字符（除换行）	<code>\a.c\</code>	<code>\abc\</code> , <code>\a1c\</code> , <code>\a c\</code>
<code>\d</code>	数字	<code>\d\</code>	<code>\0-9\</code>
<code>\D</code>	非数字	<code>\D\</code>	<code>\a, @, 空格\</code>
<code>\w</code>	单词字符	<code>\w\</code>	<code>\a-z, A-Z, 0-9, _\</code>
<code>\W</code>	非单词字符	<code>\W\</code>	<code>\@, #, 空格\</code>
<code>\s</code>	空白字符	<code>\s\</code>	<code>\空格, tab, 换行\</code>
<code>\S</code>	非空白字符	<code>\S\</code>	<code>\a, 1, @\</code>

CAUTION

在不同语言中，反斜杠的转义层级不同。例如在 Java 中，`\\d` 需要写成 `"\\\\d"`（两次转义）。

字符集

1	[aeiou]	匹配任意元音字母	text
2	[0-9]	匹配任意数字（等同于 <code>\\d</code> ）	
3	[a-zA-Z]	匹配任意字母	
4	[^0-9]	匹配非数字（ <code>^</code> 在开头表示否定）	
5	[abc^]	匹配 <code>a</code> , <code>b</code> , <code>c</code> , <code>^</code> （ <code>^</code> 不在开头就是普通字符）	

量词

量词	含义	示例	匹配次数
<code>*`</code>	0 次或多次	<code>ab*c`</code>	<code>`ac, abc, abbc, abbbc...`</code>
<code>+`</code>	1 次或多次	<code>ab+c`</code>	<code>`abc, abbc, abbbc...`</code>
<code>?`</code>	0 次或 1 次	<code>ab?c`</code>	<code>`ac, abc`</code>
<code>{n}`</code>	恰好 <code>n</code> 次	<code>a{3}`</code>	<code>`aaa`</code>
<code>{n,}`</code>	至少 <code>n</code> 次	<code>a{2,}`</code>	<code>`aa, aaa, aaaa...`</code>
<code>{n,m}`</code>	<code>n</code> 到 <code>m</code> 次	<code>a{2,4}`</code>	<code>`aa, aaa, aaaa`</code>

贪婪 vs 懒惰

贪婪模式（默认）：尽可能多匹配

1	<code>.*</code>	匹配尽可能多的字符	text
2	<code>.*+</code>	匹配尽可能多的字符	
3	<code>.{2,5}</code>	匹配2到5个，优先5个	

懒惰模式（加 `?`）：尽可能少匹配

1	<code>.*?</code>	匹配尽可能少的字符	text
2	<code>.*+?</code>	匹配尽可能少的字符	
3	<code>.{2,5}?</code>	匹配2到5个，优先2个	

示例：

1	文本：	<code><div>hello</div><div>world</div></code>	text
2			
3	贪婪：	<code><div>.*</div></code> 匹配： <code><div>hello</div><div>world</div></code>	
4	懒惰：	<code><div>.*?</div></code> 匹配： <code><div>hello</div></code> 和 <code><div>world</div></code>	

TIP

提取 HTML 标签内容时，通常使用懒惰模式，避免匹配过多内容。

锚点

锚点	含义	示例
<code>^</code>	字符串开头	<code>^Hello`</code> 匹配以 Hello 开头的字符串
<code>\$</code>	字符串结尾	<code>World\$`</code> 匹配以 World 结尾的字符串
<code>\b</code>	单词边界	<code>`\bcata\b`</code> 匹配独立的 cat
<code>\B</code>	非单词边界	<code>`\Bcata\B`</code> 匹配 embedded 中的 cat

注意：在多行模式下，`^` 和 `$` 匹配每行的开头和结尾。

分组与捕获

1	<code>(...)</code>	捕获分组	text
2	<code>(?:...)</code>	非捕获分组	
3	<code>(?P<name>...)</code>	命名分组（Python等）	
4	<code>(?<name>...)</code>	命名分组（.NET, Java）	

示例：

1	<code>(\d{4})-(\d{2})-(\d{2})</code>	匹配日期并捕获年月日	text
2			
3	分组1：	年	
4	分组2：	月	
5	分组3：	日	

反向引用

1	<code>(\\w+)\\s+\\1</code>	匹配重复的单词, 如 "hello hello"	text
2			
3	<code>\\1</code>	引用第一个分组	
4	<code>\\2</code>	引用第二个分组	

选择 (或)

1	<code>cat dog</code>	匹配 cat 或 dog	text
2	<code>(red blue)</code>	匹配 red 或 blue	

3 高级特性

3.1 零宽断言 (Lookaround)

零宽断言不消耗字符, 只判断位置。

先行断言 (Lookahead)

肯定先行 `(?=...)`: 后面必须匹配

1	<code>\\d+(?=px)</code>	匹配后面跟着px的数字	text
2		"100px" 匹配 "100"	
3		"100em" 不匹配	

否定先行 `(?!...)`: 后面不能匹配

1	<code>\\d+(?!px)</code>	匹配后面不跟px的数字	text
2		"100em" 匹配 "100"	
3		"100px" 不匹配	

后行断言 (Lookbehind)

肯定后行 `(?<=...)`: 前面必须匹配

1	<code>(?<=\\\$)\\d+</code>	匹配\$后面的数字	text
2		"\$100" 匹配 "100"	
3		"€100" 不匹配	

否定后行 `(?<!=...)`: 前面不能匹配

1	<code>(?<!\\\$)\\d+</code>	匹配\$前面的数字	text
2		"€100" 匹配 "100"	
3		"\$100" 不匹配	

NOTE

并非所有正则引擎都支持后行断言。JavaScript 在 ES2018 之前不支持, Python 和 Java 支持。

3.2 条件表达式

```
1 (? (condition)yes|no) text
2
3 如果condition匹配, 则尝试yes, 否则尝试no
```

示例:

```
1 (<)?\w+(?(1)>) text
2
3 如果有左尖括号, 必须有右尖括号
4 匹配: <tag>, tag
5 不匹配: <tag, tag>
```

3.3 原子分组

```
1 (?>...) text
2
3 一旦匹配成功, 不回溯
```

用途: 优化性能, 防止灾难性回溯

```
1 (?>a+)b text
2
3 对 "aaaaaab" 快速失败, 而不是尝试所有可能
```

3.4 递归匹配

某些引擎 (PCRE) 支持递归:

```
1 (\((?:[^(]*|(?R))*\)) text
2
3 匹配平衡的括号
4 匹配: ((())), (a(b)c)
```

CAUTION

递归匹配功能强大但复杂, 且不是所有引擎都支持。对于嵌套结构, 建议使用专门的解析器。

4 常用正则模式

4.1 数据验证

邮箱地址

```
1 ^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$ text
2
3 解释:
4 ^      开头
5 [A-Za-z0-9._%+-]+ 用户名 (允许字母、数字、特定符号)
6 @      @符号
7 [A-Za-z0-9.-]+   域名
```

8	\\.	点号
9	[A-Za-z]{2,}	顶级域名（至少2个字母）
10	\$	结尾

NOTE

这个正则能覆盖大部分常见邮箱，但不符合 RFC 5322 完整规范。生产环境建议使用专门的验证库。

手机号码（中国）

1	^1[3-9]\\d{9}\$		text
2			
3	解释：		
4	^	开头	
5	1	以1开头	
6	[3-9]	第二位是3-9	
7	\\d{9}	后面9位数字	
8	\$	结尾	

身份证号（中国）

1	^\\d{17}[\\dXx]\$		text
2			
3	解释：		
4	^	开头	
5	\\d{17}	17位数字	
6	[\\dXx]	最后一位是数字或X/x	
7	\$	结尾	

TIP

实际应用中还需要校验校验码和地区编码，仅靠正则不够。

URL

1	^https?:\\/[\\/[\\^\\s]+\$		text
2			
3	或者更严格的版本：		
4	^https?:\\/[\\/[a-zA-Z0-9.-]+(:\\d+)?(\\/[a-zA-Z0-9._~:\\/\\?#\\[\\]@!\$&'()*+,-;=\\?])?\$		

IPv4 地址

1	^(?:25[0-5] 2[0-4]\\d [01]?\\d\\d?)\\.){3}(?:25[0-5] 2[0-4]\\d [01]?\\d\\d?)\$		text
2			
3	解释：		
4	25[0-5]	250-255	
5	2[0-4]\\d	200-249	
6	[01]?\\d\\d?	0-199	
7	\\.	点号	
8	重复3次 + 最后一段		

CAUTION

IP 地址的正则很复杂，建议使用专门的验证函数。

4.2 文本提取

HTML 标签内容

```
1 <([a-zA-Z][a-zA-Z0-9]*)[>]*>(.*?)<\/\1>
2
3 捕获组1: 标签名
4 捕获组2: 标签内容
```

NOTE

解析 HTML 建议使用专门的解析器（如 Jsoup、BeautifulSoup），正则只能处理简单场景。

日志中的 IP 地址

```
1 \\b(?:\\d{1,3}\\\\.){3}\\d{1,3}\\b
```

JSON 值提取

```
1 "key"\\s*:\\s*"([^"]*)"
2
3 捕获组1: key对应的值
```

4.3 数据清洗

去除 HTML 标签

```
1 <[>]+>
2
3 替换为空字符串
```

去除多余空白

```
1 \\s+
2
3 替换为单个空格
```

提取电话号码

```
1 \\b\\d{3}[-.]?\\d{3}[-.]?\\d{4}\\b
2
3 匹配: 123-456-7890, 123.456.7890, 1234567890
```

5 各语言中的使用

5.1 Python

```
1 import re
2
3 # 编译正则
```

 Python

```
4 pattern = re.compile(r'\\d+')
5
6 # 查找所有匹配
7 matches = pattern.findall('abc123def456')
8 # ['123', '456']
9
10 # 搜索第一个匹配
11 match = pattern.search('abc123def')
12 if match:
13     print(match.group()) # '123'
14
15 # 替换
16 result = pattern.sub('*', 'abc123def456')
17 # 'abc*def*'
18
19 # 分割
20 parts = re.split(r'\\d+', 'abc123def456ghi')
21 # ['abc', 'def', 'ghi']
22
23 # 命名分组
24 pattern = re.compile(r'(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})')
25 match = pattern.match('2024-01-15')
26 print(match.group('year')) # '2024'
27 print(match.group('month')) # '01'
```

TIP

Python 中使用原始字符串 `r'...'` 可以避免反斜杠转义问题。

5.2 JavaScript

```
1 // 创建正则 ⚡ JavaScript
2 const pattern = /\\d+/g;
3
4 // 测试
5 pattern.test('abc123'); // true
6
7 // 匹配
8 'abc123def456'.match(/\\d+/g); // ['123', '456']
9
10 // 搜索
11 const match = /\\d+/.exec('abc123def');
12 console.log(match[0]); // '123'
13
14 // 替换
15 'abc123def456'.replace(/\\d+/g, '*'); // 'abc*def*'
16
17 // 分割
18 'abc123def456'.split(/\\d+/); // ['abc', 'def', '']
19
20 // 命名分组 (ES2018+)
21 const datePattern = /(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})/;
```

```
22 const match = datePattern.exec('2024-01-15');
23 console.log(match.groups.year);    // '2024'
```

5.3 Java

```
1  import java.util.regex.*;
2
3  // 编译正则（推荐复用）
4  Pattern pattern = Pattern.compile("\\\\d+");
5
6  // 完全匹配
7  boolean match = Pattern.matches("\\\\d+", "123"); // true
8
9  // 查找
10 Matcher matcher = pattern.matcher("abc123def456");
11 while (matcher.find()) {
12     System.out.println(matcher.group()); // 123, 456
13 }
14
15 // 替换
16 String result = "abc123def456".replaceAll("\\\\d+", "*");
17 // "abc*def*"
18
19 // 分割
20 String[] parts = "abc123def456".split("\\\\d+");
21 // ["abc", "def", ""]
22
23 // 命名分组
24 Pattern datePattern = Pattern.compile(
25     "(?<year>\\\\d{4})-(?<month>\\\\d{2})-(?<day>\\\\d{2})"
26 );
27 Matcher dateMatcher = datePattern.matcher("2024-01-15");
28 if (dateMatcher.matches()) {
29     System.out.println(dateMatcher.group("year")); // "2024"
30     System.out.println(dateMatcher.group("month")); // "01"
31 }
```

CAUTION

Java 中反斜杠需要两次转义：`\\\\d` 表示正则中的 `\\d`。

5.4 C#

```
1  using System.Text.RegularExpressions;
2
3  // 创建正则
4  var pattern = new Regex(@"\\d+");
5
6  // 匹配
7  var matches = pattern.Matches("abc123def456");
8  foreach (Match match in matches)
```

```
9 {
10     Console.WriteLine(match.Value); // 123, 456
11 }
12
13 // 替换
14 string result = Regex.Replace("abc123def456", @"\d+", "*");
15 // "abc*def*"
16
17 // 命名分组
18 var datePattern = new Regex(@"(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})");
19 var match = datePattern.Match("2024-01-15");
20 Console.WriteLine(match.Groups["year"].Value); // "2024"
```

TIP

C# 中使用逐字字符串 `@"..."` 可以避免反斜杠转义。

6 性能优化

6.1 预编译正则

```
1 # 错误：每次循环都编译
2 for text in texts:
3     re.match(r'\d+', text) # 慢!
4
5 # 正确：预编译
6 pattern = re.compile(r'\d+')
7 for text in texts:
8     pattern.match(text) # 快!
```

 Python

避免灾难性回溯

问题示例：

```
1 (a+)+b
2
3 对 "aaaaaaaaaac" 会导致指数级回溯
```

text

解决方案：

```
1 # 使用原子分组
2 (?>a+)b
3
4 # 或使用占有量词 (PCRE)
5 a++b
6
7 # 或重写正则
8 a+b
```

text

CAUTION

灾难性回溯会导致正则匹配非常慢甚至卡死。测试时要包含失败的情况。

6.2 简化正则

```
1 # 冗余
2 [a-zA-Z0-9_]
3
4 # 简化
5 \\w
```

6.3 使用具体字符集

```
1 # 慢：过于宽泛
2 .*
3
4 # 快：具体明确
5 [^,]*
```

6.4 锚定正则

```
1 # 慢：需要尝试每个位置
2 \\d+
3
4 # 快：从开头匹配
5 ^\\d+
```

7 调试技巧

7.1 在线测试工具

- **Regex101**: <https://regex101.com/> （支持多种语言）
- **RegExr**: <https://regexpr.com/>
- **Debuggex**: <https://www.debuggex.com/> （可视化）

7.2 分步构建

1. 从简单的模式开始
2. 逐步添加复杂性
3. 每一步都测试
4. 使用注释说明各部分

```
1 # 良好的实践
2 email_pattern = re.compile(r'''
3     ^                # 开头
4     [A-Za-z0-9._%+-]+ # 用户名
5     @                # @符号
6     [A-Za-z0-9.-]+    # 域名
7     \\.              # 点号
8     [A-Za-z]{2,}      # 顶级域名
9     $                # 结尾
```

```
10 ''' , re.VERBOSE)
```

TIP

使用 `re.VERBOSE` 标志可以在 Python 中编写带注释的多行正则。

7.3 单元测试

```
1 import unittest
2
3 class TestEmailRegex(unittest.TestCase):
4     def setUp(self):
5         self.pattern = re.compile(r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
6
7     def test_valid_emails(self):
8         valid = ['user@example.com', 'test.user@domain.org']
9         for email in valid:
10             self.assertTrue(self.pattern.match(email))
11
12     def test_invalid_emails(self):
13         invalid = ['@example.com', 'user@', 'user@.com']
14         for email in invalid:
15             self.assertFalse(self.pattern.match(email))
16
17 if __name__ == '__main__':
18     unittest.main()
```

 Python

8 最佳实践总结

8.1 何时使用正则

适合：

- 简单的模式匹配
- 数据验证
- 文本提取
- 批量替换

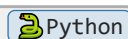
不适合：

- 解析 HTML/XML（使用解析器）
- 解析 JSON（使用 JSON 库）
- 复杂的嵌套结构
- 需要语法规则的场景

NOTE

正则不是万能的。对于结构化数据，使用专门的解析器更可靠。

8.2 代码可读性



```
1 # 不好：难以理解
2 pattern = re.compile(r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)[A-Za-z\d]{8,}$')
3
4 # 好：分解并注释
5 password_pattern = re.compile(r'''
6     ^                # 开头
7     (?=.*[A-Z])      # 至少一个大写字母
8     (?=.*[a-z])      # 至少一个小写字母
9     (?=.*\d)         # 至少一个数字
10    [A-Za-z\d]{8,}    # 至少8个字符
11    $                # 结尾
12 ''', re.VERBOSE)
```

8.3 性能考虑

- 预编译正则并复用
- 避免嵌套量词
- 使用具体字符集
- 适当使用锚点
- 测试边界情况

8.4 跨平台兼容性

- 注意不同语言的转义规则
- 测试不同引擎的行为差异
- 使用标准特性，避免引擎特有功能

本章完